

Chapitre 4

Site Interactif

JavaScript

Dr. BOUBENIA Mohamed
mo.boubenia@gmail.com

01/11/2025

Plan de cette séance

1. Rappel
2. Les Bases de la programmation côté client
3. Les Bases du Langage JavaScript
4. Manipulation du DOM
5. Le BOM (Browser Object Model)
6. Les Evénements

BOUBENIA MOHAMED

1
Rappel

BOUBENIA MOHAMED

1- Rappel

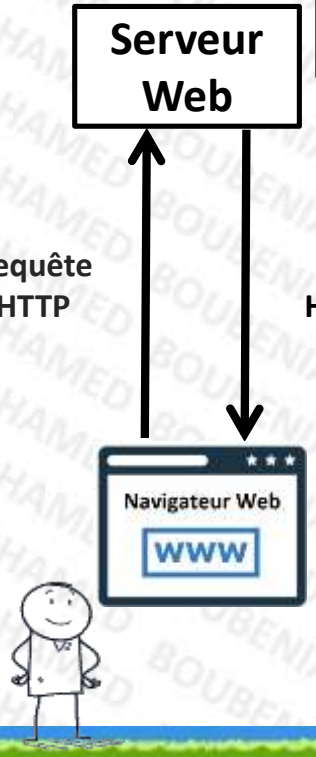
Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques

Les langage HTML et CSS suffisent pour créer des sites Web statiques :

- CV en ligne
- Site vitrine ou Portfolio (Galerie)
- Blog Statique
- Site de présentation d'un organisme
- Landing page (page de destination)
- Site de documentation technique
- Site événementiel
- Site de recette





Suffisant pour la présentation

HTML CSS

1- Rappel

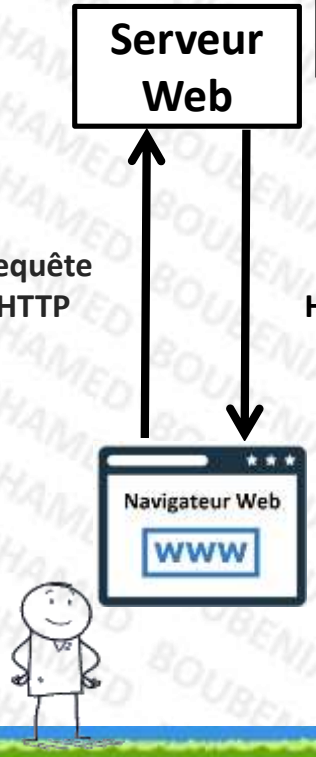
Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques

Les langage HTML et CSS suffisent pour créer des sites Web statiques :

- CV en ligne
- Site vitrine ou Portfolio (Galerie)
- Blog Statique
- Site de présentation d'un organisme
- Landing page (page de destination)
- Site de documentation technique
- Site événementiel
- Site de recette





Suffisant pour la présentation

HTML CSS

…Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

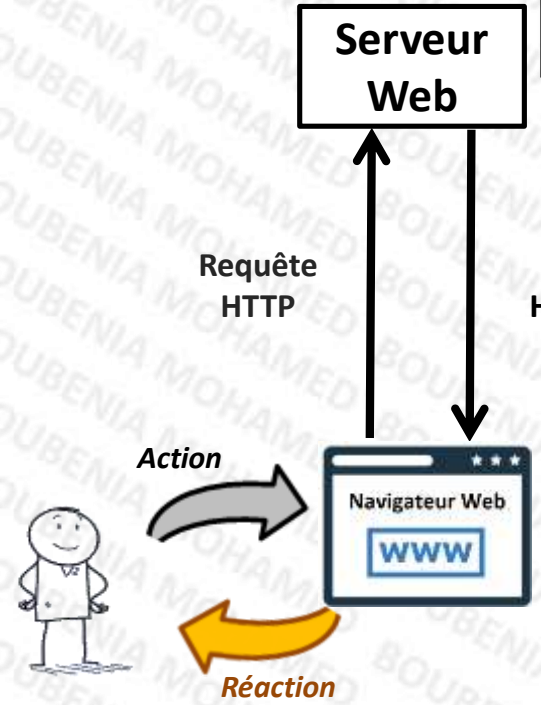
Sites Statiques **Interactif**

Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.





Suffisant pour la présentation

HTML CSS

…Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques **Interactif**

Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.

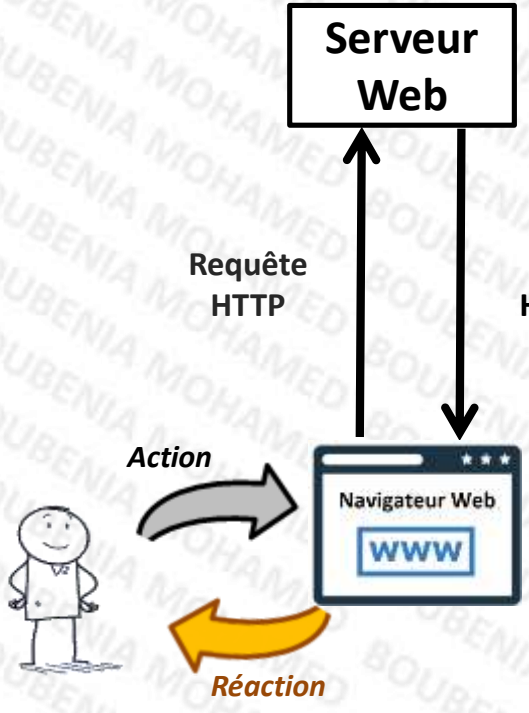
Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés



Suffisant pour la présentation

HTML **CSS**





…Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?

1- Rappel



Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques | Interactif

Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bou

- Une élément HTML s'affiche/se modifie/se m
- ou Une propriété CSS chang
- ou Un résultat d'un calcul est
- ou Suivant la véracité d'une c contenu de la page Web chan
- ou Vérification du contenu d' de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés

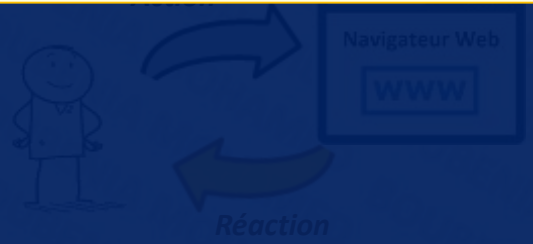
Serveur

Suffisant pour la présentation

HTML CSS

...Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?



Mais rappelez-vous

HTML n'est pas un langage de programmation

Un langage de programmation permet de donner des instructions la machine, en utilisant des variables, conditions, boucles, fonctions, etc.
HTML/CSS ne sont que des langages déclaratifs

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques **Interactif**

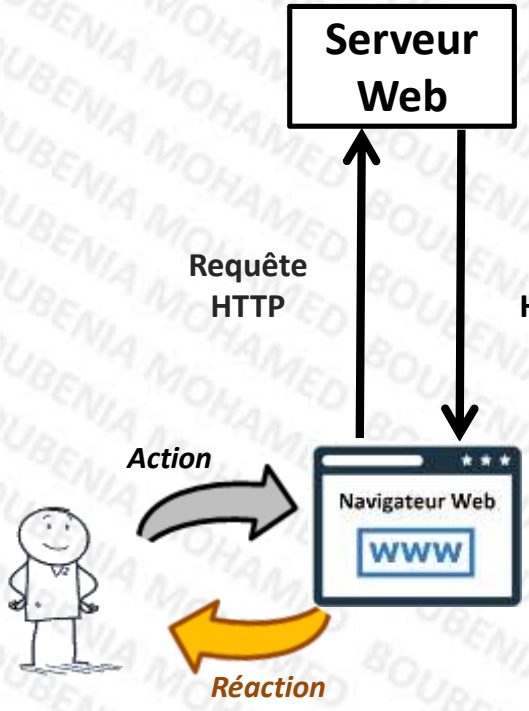
Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés





Suffisant pour la présentation

HTML CSS

JavaScript

Cette programmation se fait dans des scripts en langage JavaScript

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques **Interactif**

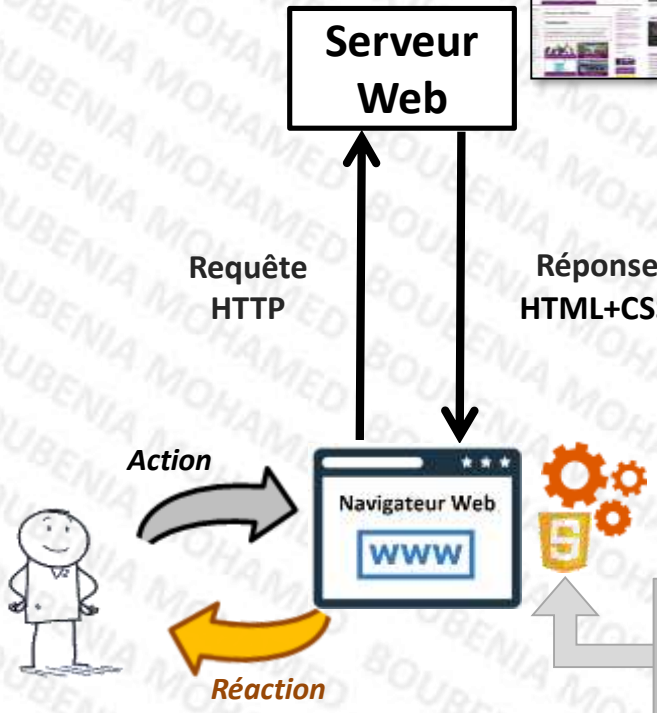
Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés





Suffisant pour la présentation

HTML CSS

JavaScript

Ces scripts sont exécutés par le navigateur au besoin (ex. un événement click se produit)

2
Les Bases de la programmation côté client

BOUBENIA MOHAMED

2- Les Bases de la programmation côté client

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>Incrémenter</button>
    </body>
</html>
```

1 Prenons l'exemple suivant. Nous avons donc un code HTML qui contient:

2 un élément H1 "Compteur=0"

3 et un élément Button (bouton) "Incrémenter"



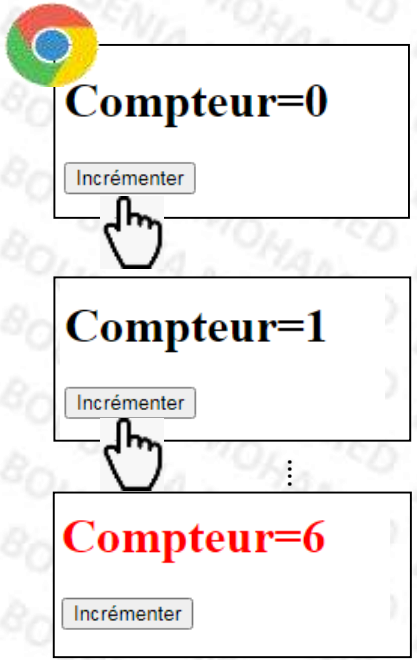
2- Les Bases de la programmation côté client

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>Incrémenter</button>
    </body>
</html>
```



2- Les Bases de la programmation côté client

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

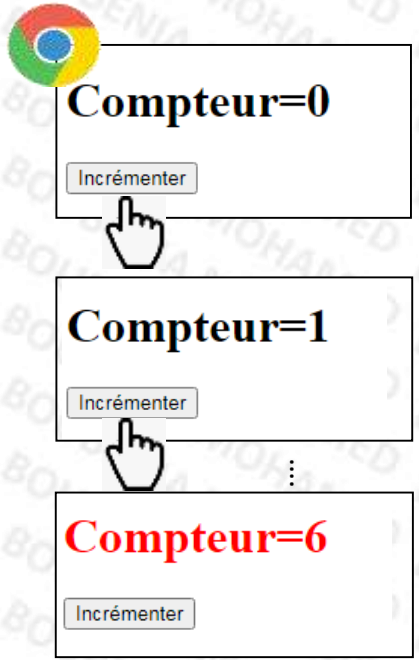
```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Script JS

```javascript
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Ce comportement doit être programmé dans un script JS (JavaScript).

2- Les Bases de la programmation côté client

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

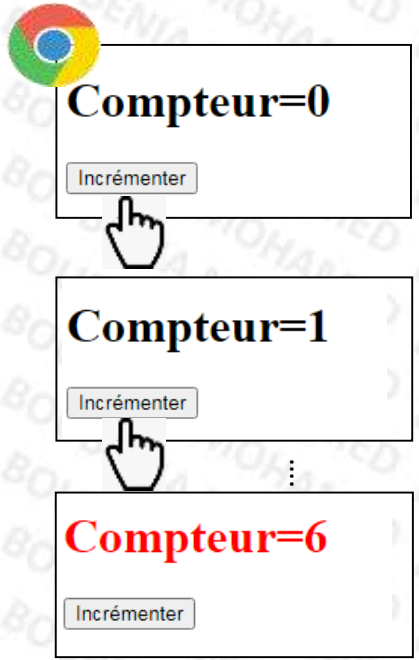
Ce scripts sera exécuté par le navigateur lors d'un click sur le bouton

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Ce comportement doit être programmé dans un script JS (JavaScript).

2- Les Bases de la programmation côté client

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

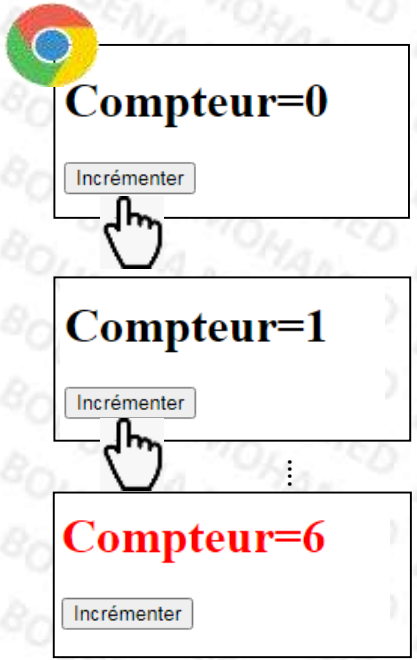
Décortiquons un peu ce comportement

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



2- Les Bases de la programmation côté client

Décortiquons un peu ce comportement

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Evénement

lorsque l'utilisateur clique sur le bouton "Incrémenter",

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

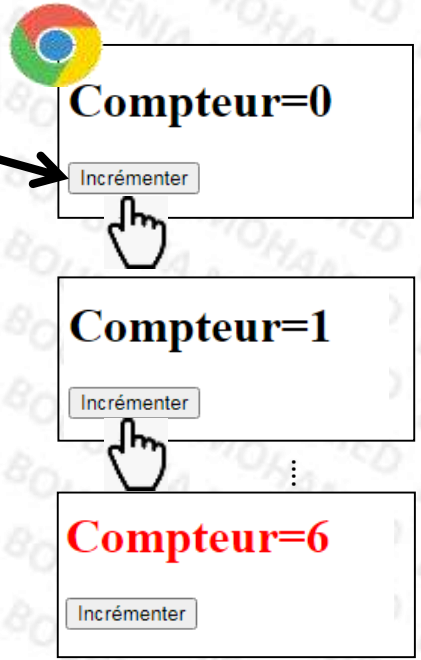
Nous avons attaché à l'élément bouton un événement (au Clic). Ca veut dire, quand il sera cliqué

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



2- Les Bases de la programmation côté client

Décortiquons un peu ce comportement

lorsque l'utilisateur clique sur le bouton "Incrémenter", **le compteur s'incrémente de 1**, et si on dépasse 5, il devient rouge.

Evénement

lorsque l'utilisateur clique sur le bouton "Incrémenter",

Traitement

le compteur s'incrémente de 1

Changement visuel

HTML

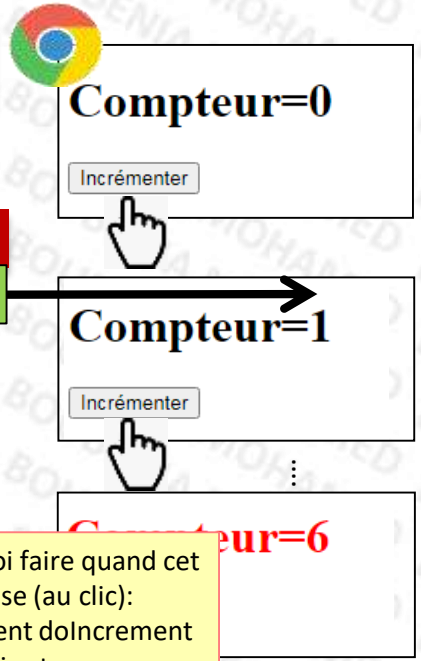
```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Nous précisons quoi faire quand cet événement se réalise (au clic):
Ici, c'est le traitement doIncrement qui est exécuté (qui est une fonction).

Puis un changement visuel est effectué, qui consiste à modifier le contenu de l'élément h1

2- Les Bases de la programmation côté client

Décortiquons un peu ce comportement

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, **et si on dépasse 5, il devient rouge.**

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

lorsque l'utilisateur clique sur le bouton "Incrémenter",

Traitement

le compteur s'incrémente de 1

Changement visuel

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

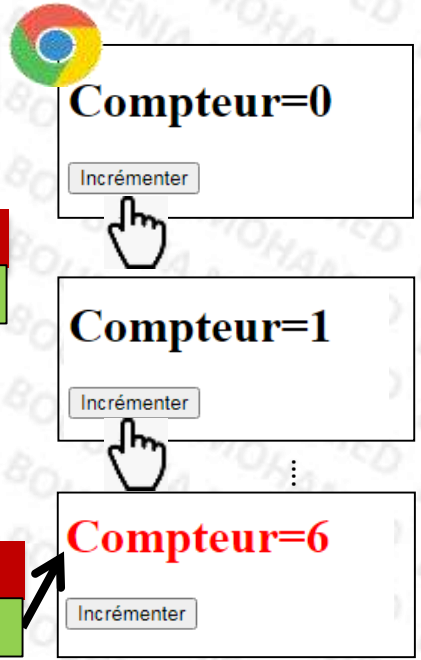
    /*Prévoir le comportement à faire quand l'événement C      aché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```

Traitement

et si on dépasse 5, il devient rouge.

Changement visuel



Nous pouvons aussi mettre une condition comme un traitement, si elle est vérifiée, on fait un changement visuel. Ici, on a changé la couleur de l'élément h1

2- Les Bases de la programmation côté client

Résumons

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

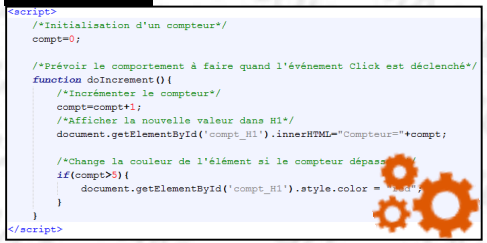
Evénement

Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

2- Les Bases de la programmation côté client

Résumons

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

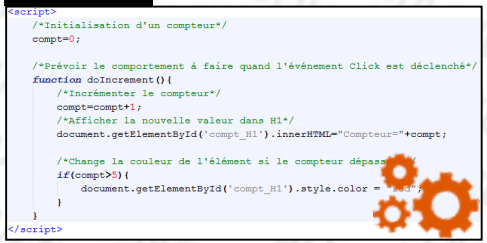
Evénement

Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation

Oui… Mais où mettre ces scripts? **Il existe 3 méthodes**

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 1: Enligne

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt H1">Compteur=0</h1>
        <button onclick="document.getElementById('compt_H1').innerHTML='Hello!!!'">
            Incrémenter
        </button>
    </body>
</html>
```

Consiste à mettre le script directement dans la balise ouvrante de l'élément concerné, en précisant l'événement.
Idéal quand il s'agit d'un petit traitement **(peu recommandé)**

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 1: Enligne

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="document.getElementById('compt_H1').innerHTML='Hello!!!'">
            Incrémenter
        </button>
    </body>
</html>
```

Méthode 2:Dans un élément <script></script> à la fin de body

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
        <script>
            /*Traitement*/
            function doIncrement(){
                /* do Increment*/
            }
        </script>
    </body>
</html>
```

Consiste à mettre le script à l'intérieur d'un élément <script></script>, et de mettre cet élément n'importe où dans le document HTML (dans le body, où dans le head)

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 1: Enligne

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="document.getElementById('compt_H1').innerHTML='Hello!!!'">
            Incrémenter
        </button>
    </body>
</html>
```

Méthode 2:Dans un élément <script></script> à la fin de body

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
        <script>
            /*Traitement*/
            function doIncrement(){
                /* do Increment*/
            }
        </script>
    </body>
</html>
```

Méthode 3: Fichier JS

```html
<!DOCTYPE html>
<html>
    <head>
        <script src="mon_script.js"></script>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

mon_script.js

```javascript
/*Traitement*/
function doIncrement(){
    /* do Increment*/
}
```

Consiste à mettre le script dans un autre fichier qui porte l'extension .js

Dans cet exemple, nous avons mis notre script dans « mon_script.js"

et nous lui avons fait référence dans notre fichier HTML, grâce à l'élément script, dans l'élément Head.

Vous avez la possibilité de charger plusieurs fichiers

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 3: Fichier JS avec URL

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
      <!-- Script externe chargé depuis une URL -->
      <script src="https://cdnjs.cloudflare.com/ajax/libs/moment.js/2.30.1/moment.min.js"></script>
      <script>
        console.log("Date actuelle :", moment().format());
      </script>
    </body>
</html>
```

Vous avez également la possibilité de faire référence à des fichiers JS sur le Web (ex. bibliothèques)

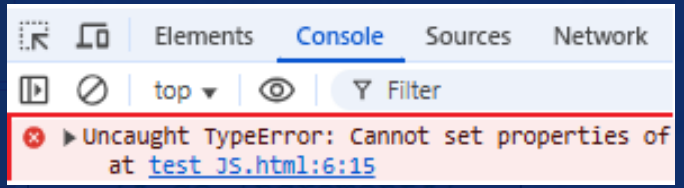
Erreur fréquente de débutant

Lorsque vous mettez votre script dans le head, il sera exécuté avant le chargement en entier du DOM (votre page)

Méthode 3: Fichier JS

```
<!DOCTYPE html>
<html>
    <head>
        <script src="script.js">
            /*Changer la couleur de H1*/
        </script>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
    </body>
</html>
```

Ce code produira une erreur, car le code JS essayera de manipuler un élément (h1) qui n'a pas encore été créé.



2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Solutions possibles

1 Vous pouvez différer l'exécution de script.js après le chargement de la page.

Méthode 3: Fichier JS

```
<!DOCTYPE html>
<html>
    <head>
        <script src="script.js" defer>
            /*Changer la couleur de H1*/
        </script>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
    </body>
</html>
```

2 Vous pouvez explicitement mettre l'élément script à la fin de body

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <script src="script.js" defer></script>
    </body>
</html>
```

Vous pouvez également utiliser un événement approprié **DOMContentLoaded**

2- Les Bases de la programmation côté client

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

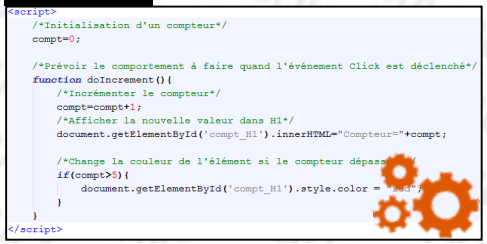
Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Nous allons décortiquer

Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

2- Les Bases de la programmation côté client

Les bases du langage JavaScript

D'abord, Il est important de connaitre le langage JavaScript (syntaxe, sémantique, etc.)

Nous allons décortiquer

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation

3- Les Bases du Langage JavaScript

C'est quoi ?

JavaScript est principalement un langage de programmation côté client.

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Historique

- JavaScript (anciennement ECMAScript) a été créé par Netscape (le fameux navigateur Web),en 1995.
- **But:** Permettre aux développeurs Web d'incorporer un code exécutable dans leurs pages Web.
- **Effet:** Une page Web pouvait donc contenir un script en JavaScript, qui est exécuté quand l'utilisateur visite la page (interactivité ou traitement).

3- Les Bases du Langage JavaScript

Caractéristiques

- **Langage multi-paradigmes**
  Impérative, Fonctionnelle, POO.
- **Syntaxe**
  Fortement inspirée de C++ et Java.
  Ne pas confondre avec Java (aucun lien, même si similitude dans le nom).

…Mais, Différences majeures avec des langages comme C++ et Java ! :

- **Langage non compilé, mais interprété:**
  Il n'y a pas d'étape de compilation. A la place, un interpréteur (présent dans le navigateur (ex. V8, SpiderMonkey, etc.)), lit le code JS, interprète chaque ligne, et l'exécute (compilation Just-In-Time (JIT)).
- **Langage dynamique et faiblement typé**
  Une variable peut à tout moment changer de type. JS convertit automatiquement les types si nécessaire (ex. 2+ "2" donne "22").
- **Garbage Collector**
  Intégré dans le moteur JavaScript pour la gestion de la mémoire.
- **Spécification:** ECMAScript (ES) Specification.

Versions ECMAScript

Edition	Nom Officiel	Date publication
ES16	ES2025	June 2025
…	…	…
ES10	ES2019	Summer 2019
ES9	ES2018	June 2018
ES8	ES2017	June 2017
ES7	ES2016	June 2016
ES6	ES2015	June 2015
ES5.1	ES5.1	June 2011
ES5	ES5	December 2009
ES4	ES4	Abandoned
ES3	ES3	December 1999
ES2	ES2	June 1998
ES1	ES1	June 1997

3- Les Bases du Langage JavaScript

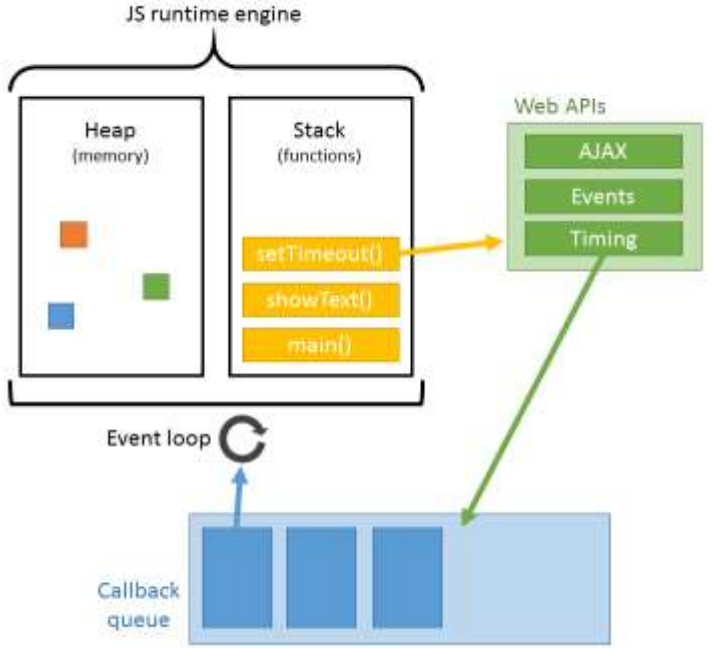
Caractéristiques

Single Threaded (Une seule file d'exécution) avec Event-Loop

JS s'exécute sur un seul thread principal. Cela signifie qu'il ne peut pas exécuter plusieurs instructions à la fois (pas de multithreading natif). Mais, il n'est pas bloquant grâce à son modèle asynchrone

4 composant principaux:

1. **Event Loop**
2. **Stack (pile d'appels): Syncrhone**
3. **Heap (tas de mémoire):**
4. **Callback Queue (file): Asynchrone**



Déroulement:

1. L'Event Loop (la boucle d'événements, sur le thread principal) exécute le code JavaScript ligne par ligne dans la Call Stack.
2. Les objets et variables sont stockés dans la Heap memory.
3. Lorsqu'une opération asynchrone est rencontrée (par ex. setTimeout, appel à une API, événement DOM), elle est déléguée à l'environnement du navigateur (les Web APIs).
4. Quand cette opération est terminée, son callback est placé dans la Callback Queue (ou Task Queue).
5. L'Event Loop surveille en permanence la Call Stack : dès qu'elle est vide, il y récupère la prochaine tâche de la Callback Queue pour l'exécuter.

3- Les Bases du Langage JavaScript

Familiarités avec les langages de programmation

Contrairement au langage C ou Java, en JavaScript il n'y a pas de notion de fonction principale (la fameuse fonction main()).

A la place, le navigateur interprète et exécute le code de haut vers le bas, et compris les fichiers JS externes.

Comme avec tout langage de programmation, en JavaScript, il y a aussi la console qu'on peut utiliser pour afficher (on écrit: console.log("text à afficher")).

Le console est accessible dans n'importe quel navigateur. Par exemple, ici, on a ouvert la page maPage.html, puis les outils de développement, puis l'onglet Console.

Recommandé lorsqu'on fait des teste (ex. les valeurs des variables).

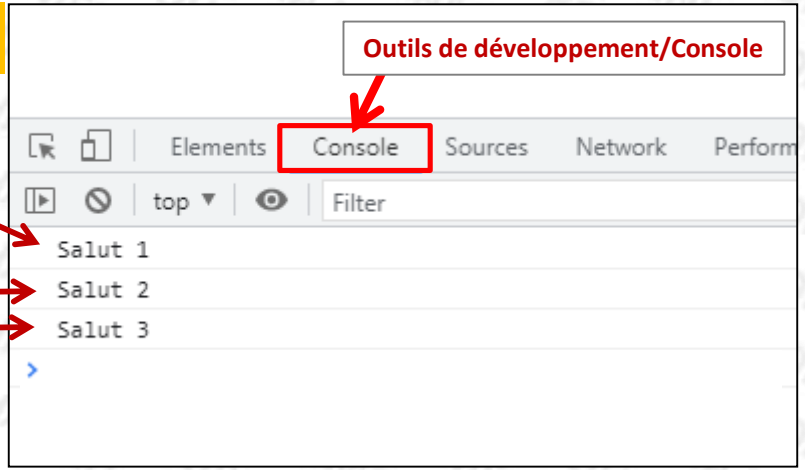
maPage.html

```
<!DOCTYPE html>
<html>
    <head>
        <script src="mon_script.js"></script>
    </head>
    <body>
        <h1>Titre1</h1>
        <script>
            console.log("Salut 2");
            console.log("Salut 3");
        </script>
    </body>
</html>
```

Mon_script.js

```
console.log("Salut 1");
```

Console.log()



3- Les Bases du Langage JavaScript

Familiarités avec les langages de programmation

D'autres soties possibles

innerHTML/innerText

```
<p></p>
<script>
    elt = document.getElementsByTagName("p")[0];
    elt.innerHTML="Hello World"
</script>
```

Hello World

Alert()

```
<script>
    alert("Hello World");
</script>
```



3- Les Bases du Langage JavaScript

Noyau du langage JS

1. Syntaxe
2. Variables et Constantes
3. Les opérateurs
4. Les conditions
5. Les boucles
6. Les fonctions
7. Portée d'une variable

BOUBENIA MOHAMED

3- Les Bases du Langage JavaScript

- Syntaxe proche de celle du langage C: quasiment les mêmes opérateurs, les mêmes instructions.
- Toutes les instructions doivent se terminer par un point virgule (;) bien que ça reste facultatif.

```
console.log("Hello World");
```

Les affectations

Signe égale: Ici on a fait un calcul, et le résultat est affecté à une variable "a"

```
a = 1 + 2;
```

Bloc de code

Exemple: Fonction, conditions, etc

```
{
    // Intruction 1
    // Intruction 2
}
```

Les commentaires

// pour chaque ligne

```
// console.log("Hello World");
```

Sur plusieurs lignes : commence par /* et se termine par */

```
/*
    console.log("Hello World");
    console.log("Hello World");
*/
```

3- Les Bases du Langage JavaScript

Noyau du langage JS | 2. Variables et Constantes | Les Types primitifs

String
number
boolean
null

```
// Initialisation des variables
nom = 'JOHN';  // chaine de caractère
prenom = "DOE"; // chaine de caratère
age = 30; // entier
note = 15.5 // réel
estAdulte = true; // booléen
x = null; // valeur nulle
const pi = 3.14;
// Appel des variables
console.log(nom, prenom);  // Affichage multiple
console.log("Nom="+nom+", Prénom="+prenom);  // Affichage élaboré
console.log(`Bonjour ${nom}`); // Affichage formaté (backstick)
console.log(age);
console.log(note);
console.log(estAdulte);  // Affichage multiple
console.log(x);
console.log(pi);
console.log(estadulte);  // Erreur (var non trouvée) car sensibilité à la casse
// l'interpéteur s'arrête à la rencotre d'une erreur
/*
    le code ici sera ignoré
*/
```

> Console

```
JOHN DOE
Nom=JOHN, Prénom=DOE
Bonjour JOHN
30
15.5
true
null
3.14
Uncaught ReferenceError: estadulte is not defined
    at test.html:31
>
```

3- Les Bases du Langage JavaScript

Noyau du langage JS | 2. Variables et Constantes | Les Types Complexes

Les Tableaux

```
//tableau d'entier
monTableau1 = new Array(1,2,3,4,5);
//une autre manière
monTableau2 = [1,2,3,4,5];
console.log(monTableau1);
console.log(monTableau2);

//tableau de chaine de caractère
monTableau3 = ['A','B','test','D'];
console.log(monTableau3);

//tableau avec un contenu divers
monTableau4 = ['A','B',10,true];
console.log(monTableau4);
// Accès au contenu du tablau
console.log(monTableau4[0]);
console.log(monTableau4[3]);

// Modification
monTableau4[0] = 'BOB';
console.log(monTableau4);
```

> Console

```
▸ (5) [1, 2, 3, 4, 5]
▸ (5) [1, 2, 3, 4, 5]
▸ (4) ['A', 'B', 'test', 'D']
▸ (4) ['A', 'B', 10, true]
A
true
▸ (4) ['BOB', 'B', 10, true]
```

Destructuration

```
monTableau1 = [1,2,3,4,5]
const [x,y,z]= monTableau1
// Donne 1, 2, 3
```

Méthodes utiles

```
// ajouter à la fin
monTableau1.push(6);

// Supprime le dernier élément
monTableau1.pop();

// vérifie si le tableau contien un élément
monTableau1.includes(2)
// donne true

// Retourne le premier élément
// qui satisfait une condition
res = monTableau1.find(n => n > 3);
// donne 4

// filter: Sélectionner certains éléments
grands = monTableau1.filter(n => n > 3);
  // donne [4, 5]

// map: Transformer les valeurs
doubles = monTableau1.map(n => n * 2);
// donne [2, 4, 6, 8, 10]

// reduce: Calcul (somme, produit, etc.)
// acc → accumulateur (initialisé à 0 ici)
// val → valeur courante
somme = monTableau1.reduce
                ((acc, val) => acc + val, 0);
// donne 15
```

3- Les Bases du Langage JavaScript

Les Objets

Dans JS pratiquement tout repose sur les objets: les tableaux, les fonctions, même les classes sont en fait des objets.

```
// déclaration de l'objet person
const person = {
    nom : 'JOHN',
    prenom: 'DOE',
    age: 30,
    centresInterets: ['sport', 'films', 'peinture'],
    adresse: {
        cite: '5 Juillet',
        commune: 'Bab-Ezzouar',
        ville: 'Alger'
    }
};
console.log(person);
console.log(person.nom);      // accès à nom
console.log(person["age"]); // accès à age
```

> Console

```
▶ {nom: 'JOHN', prenom: 'DOE', age: 30, centresInterets: Array(3), adresse: {…}}
JOHN
30
>
```

```
// Rajouter une propriété
person.metier="Ingenieur";

// Modifier une propriété
person.nom="BOB";

// Supprimer une propriété
delete person.metier;
```

Destructuration

```
const person = {
    nom: "JOHN",
    prenom: "DOE"
};

const { nom, prenom } = person;
```

3- Les Bases du Langage JavaScript

Tableau d'Objets

```
const etudiants = [
    {
        id: 1,
        nom: 'nom1',
        prenom: 'prenom1'
    },
    {
        id: 2,
        nom: 'nom2',
        prenom: 'prenom2'
    },
    {
        id: 3,
        nom: 'nom3',
        prenom: 'prenom3'
    }
];
console.log(etudiants);
console.log(etudiants[0].nom);

const etudiantJSON = JSON.stringify(etudiants);
console.log(etudiantJSON);
```

```
▼(3) [{…}, {…}, {…}]
  ▶0: {id: 1, nom: 'nom1', prenom: 'prenom1'}
  ▶1: {id: 2, nom: 'nom2', prenom: 'prenom2'}
  ▶2: {id: 3, nom: 'nom3', prenom: 'prenom3'}
    length: 3
  ▶[[Prototype]]: Array(0)
nom1
[{"id":1,"nom":"nom1","prenom":"prenom1"},{"id":2,"nom":"nom2","prenom":"prenom2"},{"id":3,"nom":"nom3","prenom":"prenom3"}]
>
```

Le tableau d'objets peut aussi être transformé en objet JSON (JavaScript Objet Notation), qui est un format léger, et très utilisé pour les échanges client-serveur.
On le reverra dans la partie Site Web Dynamique

3- Les Bases du Langage JavaScript

Arithmétiques

```
val = 1+2; // Addition
val = 2-1; // Soustraction
val = 2*2; // Multiplication
val = 4/2; // Division
val = 4%2; // Reste Division
val++; // Incrémente de 1
val--; // Décrémente de 1
```

Opérateurs combiné

```
x += 3; // x=x+3
x -= 2; // x=x-2
x *= 4; // x=x*4
x /= 2; // x=x/2
x %= 3; // x=x%3
x **= 2;// x=x²
```

Logiques

```
val1=true;
val2=false;

console.log(val1 && val1); // &&: le ET logique
console.log(val1 && val2);
console.log(val1 || val2); // ||: le OU logique
console.log(!val1);        // ! le NON logique
```

```
true
false
true
false
>
```

Rationnels

```
console.log(2>1);
console.log(2<1);
console.log(2<=1);
console.log(2>=1);
console.log(2==1); // égalité
console.log(2!=1); // inégalité
console.log(5=="5"); // true
console.log(5==="5"); // false
```

```
true
false
false
true
false
true
>
```

3- Les Bases du Langage JavaScript


```
val1=1;
val2=2;
val3=1;

// Condition if avec une seule instruction
if(val2>val1) console.log('val2 est supérieure à val1');

// Condition if avec plusierus instructions
if(val2>val1) {
    console.log('Hello World !');
    console.log('val2 est supérieure à val1');
}

// Condition if et else
if(val2>val1)
    console.log('val2 est supérieure à val1');
else
    console.log('val2 est inférieure à val1');


// Condition if et else if
if(val1>val3)
    console.log('val1 est supérieure à val3');
else if(val1<val3)
    console.log('val1 est inférieure à val3');
else
    console.log('val1 est égale à val3');
```

```
val2 est supérieure à val1
Hello World !
val2 est supérieure à val1
val2 est supérieure à val1
val1 est égale à val3
>
```

```
// Notation Ternaire
val1=1;
(val1==1)? res=true:res=false;
// true
```

3- Les Bases du Langage JavaScript


```
jour = 3;

switch (jour) {
  case 1:
    nomJour = "Lundi";
    break;
  case 2:
    nomJour = "Mardi";
    break;
  case 3:
    nomJour = "Mercredi";
    break;
  default:
    nomJour = "Jour inconnu";
}

console.log(nomJour);
// Mercredi
```

BOUBENIA MOHAMED

3- Les Bases du Langage JavaScript


```
// tableau d'étudiants
const etudiants = [
    {
        id: 1,
        nom: 'nom1',
        prenom: 'prenom1'
    },
    {
        id: 2,
        nom: 'nom2',
        prenom: 'prenom2'
    },
    {
        id: 3,
        nom: 'nom3',
        prenom: 'prenom3'
    }
];
```

```
// Boucle: for
for(let i=0; i<=2; i++){
    console.log(etudiants[i].nom);
}

// Boucle: While
i=0;
while(i<etudiants.length){
    console.log(etudiants[i].nom);
    i++;
}

// Boucle: for..of
for(etudiant of etudiants){
    console.log(etudiant.nom);
}

// Boucle forEach
etudiants.forEach(function(etudiant){
    console.log(etudiant.nom);
});
```

```
nom1
nom2
nom3
>
```

forEach ne peut pas être interrompue avec **break** (utilisé pour interrompre une boucle), et ne peut pas sauter une iteration avec **continue**. Elle ne peut pas faire des **return** non plus.

3- Les Bases du Langage JavaScript

Noyau du langage JS | 6. Les Fonctions

```
// Déclaration d'une fonction sans paramètres
function doCalcul0(){
    console.log(5*10);
}
// Déclaration d'une fonction avec paramètres
function doCalcul1(val1, val2){
    console.log(val1+val2);
}
// Déclaration d'une fonction avec un retour
function doCalcul2(val1, val2){
    return val1*val2;
}

// Appel
doCalcul0();
doCalcul1(1,2);
console.log(doCalcul2(1,2));
```

```
50
3
2
>
```

```
const etudiants = [
    {
        id: 1,
        nom: 'nom1',
        prenom: 'prenom1'
    },
    {
        id: 2,
        nom: 'nom2',
        prenom: 'prenom2'
    },
    {
        id: 3,
        nom: 'nom3',
        prenom: 'prenom3'
    }
];

// Le passage de paramètre se fait par référence
// et non pas par valeur
function doCalcul3(objetEtudiant){
    // modifiera etudiants[1]
    objetEtudiant.id=objetEtudiant.id*10;
}
doCalcul3(etudiants[1]);
console.log(etudiants[1].id);
```

```
20
>
```

3- Les Bases du Langage JavaScript

Noyau du langage JS | 6. Les Fonctions

Définir une valeur par défaut pour le paramètre

```
function doCalcul3(val=1){
    return val*3
}

console.log(doCalcul3());
// Affiche 3
```

- Hoistée
- Propre this
- Réutilisable

Fonction anonyme et invoquée immédiatement

```
(function(val=1) {
  console.log(val*3);
})();
// Affiche 3
```

- Non Hoistée
- Propre this
- Non Réutilisable
- Invocation immédiate
- Protège le code

Fonction fléchée (Arrow Function)

```
doCalcul3 = (val=1) =>{
    return val*3
}

console.log(doCalcul3());
// Affiche 3
```

- Non Hoistée
- This du parent
- Réutilisable
- Utile pour les callbacks

3- Les Bases du Langage JavaScript

Un callback est une fonction passée en paramètre à une autre fonction, et appelée plus tard.

```
function processData(data, callback) {
  console.log("Traitement des données...");
  callback(data);
}

processData("Bonjour", function(result) {
  console.log("Résultat :", result);
});
```

```
function processData(data, callback) {
  console.log("Traitement des données...");
  callback(data);
}

f = (result) => {
    console.log("Résultat :", result);
}

processData("Bonjour", f);
```

3- Les Bases du Langage JavaScript

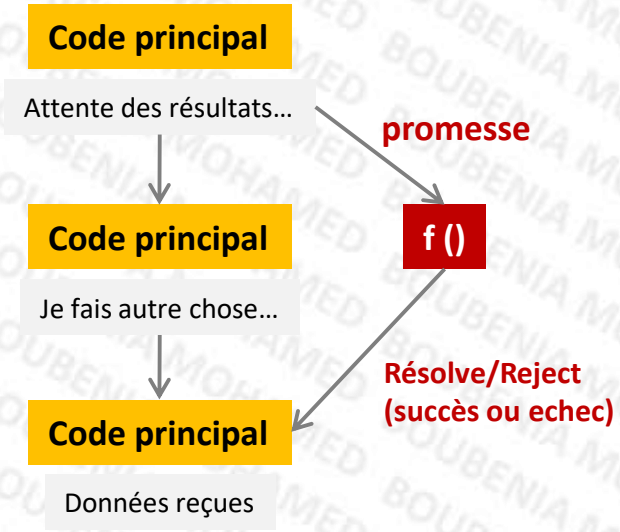
Une Promise est un objet qui représente une opération asynchrone.
C'est comme un contrat : « Je te promets que je te rendrai un résultat (ou une erreur), mais plus tard. »

```
// soit f() une fonction bloquante
f = (resolve, reject)=>{
    setTimeout(() => resolve("Données reçues !"), 1000);
}

// je peux l'exécuter de manière asynchrone non bloquante
const fetchData = new Promise((resolve, reject) => {
  console.log("Attente des résultat...");
  f(resolve, reject);
});

// je précise quoi faire quand elle se débloque
fetchData
  .then(result => console.log(result))
  .catch(error => console.error(error));

// je précise quoi faire en parallèle
console.log("Pendant que j'attends, je fais autre chose");
```



3- Les Bases du Langage JavaScript

Une façon plus lisible d'écrire un code asynchrone.

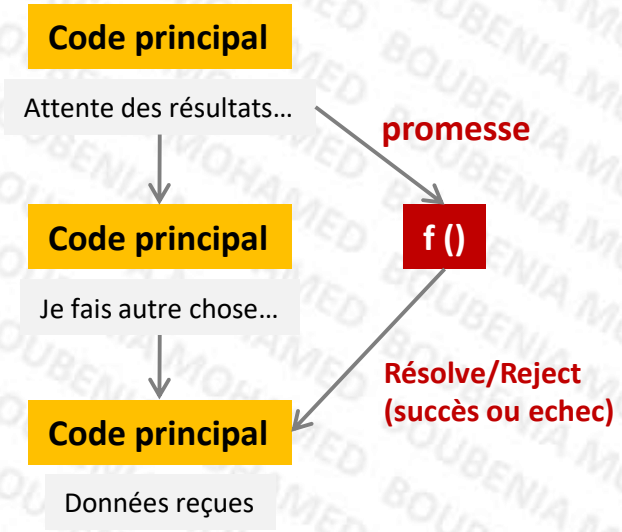
```
// soit f() une fonction bloquante
// Cette fois on met la promess dans f()
f = () => {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("Données reçues !"), 2000);
  });
};

async function fetchData() {
  try {
    console.log("Attente des résultat...");
    const result = await f(); // on appelle f() pour obtenir la Promise

    console.log(result);
  } catch (error) {
    console.error(error);
  }
}

fetchData();

// je précise quoi faire en parallèle
console.log("Pendant que j'attends, je fais autre chose");
```



3- Les Bases du Langage JavaScript


```
a = 'a'; // implicitement globale
b = 'b'; // implicitement globale

if(2>1){
    let d = 'd'; // let: portée dans le bloc courant
    e = 'e'; // implicitement globale
}

function myFunction(){
    c = 'c'; // implicitement globale
    let f= 'f'; // portée locale à la fonction
    const x= 'x';

     console.log("Appel depuis muFunction: "+a); // global
     console.log("Appel depuis muFunction: "+b); // global
     console.log("Appel depuis muFunction: "+c); // global
    // console.log("Appel depuis muFunction: "+d); // error
     console.log("Appel depuis muFunction: "+e); // global
     console.log("Appel depuis muFunction: "+f); // local
     console.log("Appel depuis muFunction: "+x); // local

}
const y = 'y';

myFunction();
console.log("Appel global: "+a); // global
console.log("Appel global: "+b); // global
console.log("Appel global: "+c); // global, seulement si appel à myFonction
//console.log("Appel global: "+d); // error: 'c' porté locale au block if
console.log("Appel global: "+e);
//console.log("Appel global: "+f); // error: 'f' porté locale à myFunction
//console.log("Appel global: "+x); // error: 'x' porté locale à myFunction
console.log("Appel global: "+y);
```

```
Appel depuis muFunction: a
Appel depuis muFunction: b
Appel depuis muFunction: c
Appel depuis muFunction: e
Appel depuis muFunction: f
Appel depuis muFunction: x
Appel global: a
Appel global: b
Appel global: c
Appel global: e
Appel global: y
```

3- Les Bases du Langage JavaScript

Initialiser une variable sans la préfixer par Let ou Const ou Var est une mauvaise pratique.
Elle aura une portée globale implicite, même dans un bloc.

```
x = 10;
```

La bonne pratique voudrait qu'on maitrise manuellement la portée de nos variable.

Pour ne pas se tromper, on utilise le mode stricte, qui nous signale les variables non définie avec Let, ou Const, ou Var:

```
"use strict";
x = 10;
```

```
Uncaught ReferenceError: x is not defined
    at test JS.html:10:6
```

Pour éviter les erreurs, on utilise donc let pour les variables, et const pour les constantes

```
"use strict";
let x = 10;
const pi = "3.14";
```

Dans les anciennes version de JS, on utilisait le mot clé var.

```
"use strict";

if(true){
    var x = 10;
}

console.log(x);
```

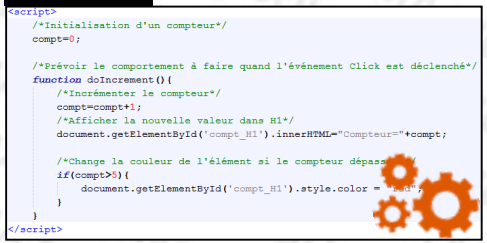
Mais le problème avec var c'est que la variable était Rehostée (hoisting), et donc sa portée devenait globale

Les bases du langage JavaScript

D'abord, Il est important de connaitre le langage JavaScript (syntaxe, sémantique, etc.)

Nous allons décortiquer

Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

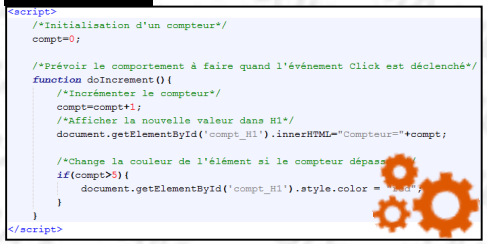
Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Nous allons décortiquer

Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

Manipulation du DOM

Il est important de comprendre le DOM, pour savoir comment sélectionner un élément HTML et comment le modifier.

Nous allons décortiquer

Changement visuel

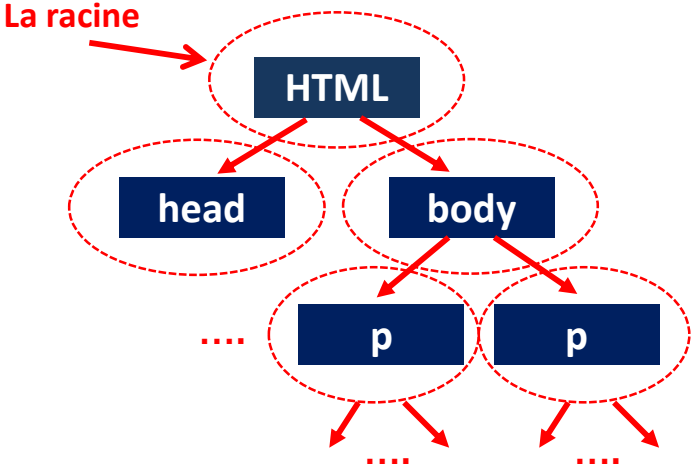
On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

4- Manipulation du DOM

Rappel: Structure d'un document HTML

Document HTML

```
<html>
    <head>
    </head>
    <body>
        <p>
            Bienvenue sur <strong>Bingo</strong>, le site
            de <em>formation</em>.
        </p>
        <p> Aujourd'hui nous allons parler de HTML. </p>
    </body>
</html>
```



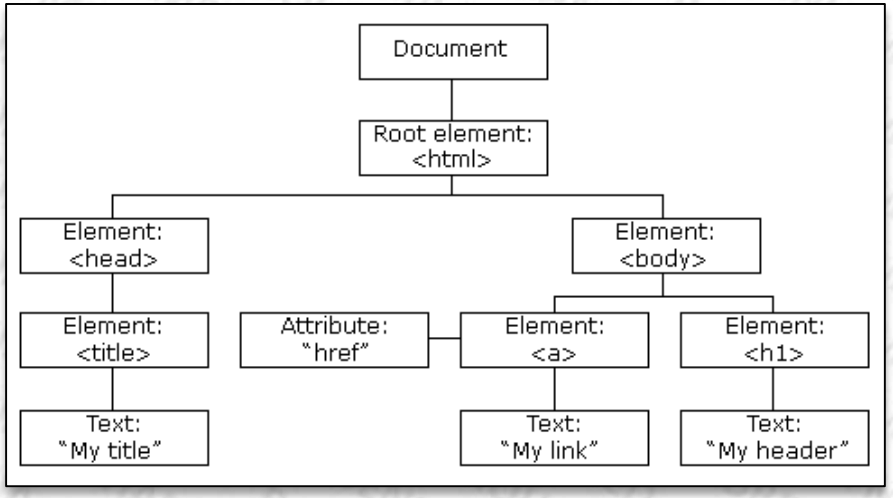
Un modèle en ARBRE

Le DOM: Document Object Model

4- Manipulation du DOM

Le DOM (Document Object Model)

Lorsque le navigateur charge une page Web,
il crée un **D**ocument **O**bjet **M**odel pour cette page
(cet objet a la structure d'un arbre).



Avec cet objet, JavaScript a tout le pouvoir dont il a besoin pour modifier dynamiquement le contenu de la page HTML:

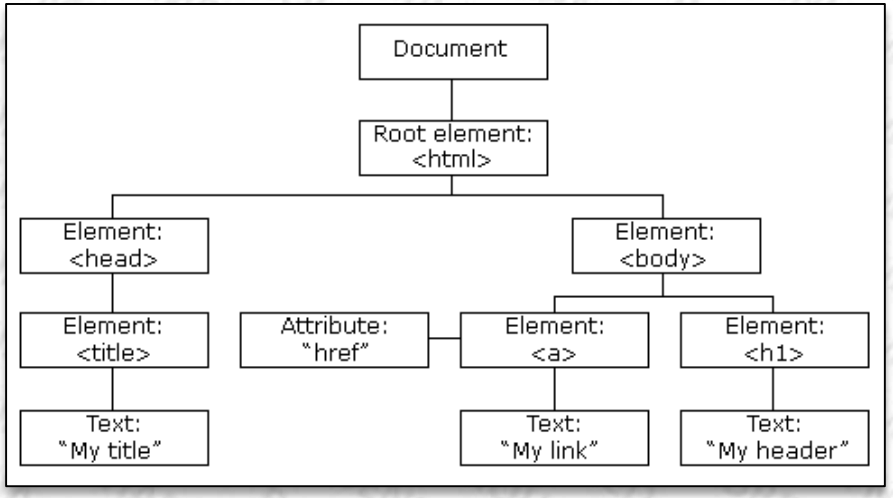
- Il peut changer tous les élément HTML sur la page.
- Il peut changer tous les attributs HTML sur la page.
- Il peut changer tous les styles CSS sur la page.
- Il peut supprimer un élément/attribut HTML.
- Il peut ajouter un nouveau élément/attribut HTML.
- Il peut réagir à tous les événements sur la page.
- Il peut créer de nouveaux événement sur le page.

BOUBENIA MOHAMED

4- Manipulation du DOM

Le DOM (Document Object Model)

Lorsque le navigateur charge une page Web,
il crée un **D**ocument **O**bjet **M**odel pour cette page
(cet objet a la structure d'un arbre).



Avec cet objet, JavaScript a tout le pouvoir dont il a besoin pour modifier dynamiquement le contenu de la page HTML:

- Il peut changer tous les élément HTML sur la page.
- Il peut changer tous les attributs HTML sur la page.
- Il peut changer tous les styles CSS sur la page.
- Il peut supprimer un élément/attribut HTML.
- Il peut ajouter un nouveau élément/attribut HTML.
- Il peut réagir à tous les événements sur la page.
- Il peut créer de nouveaux événement sur le page.

Le tout se fait généralement en 2 étapes :

1. **Retrouver l'élément HTML** — *Analogie (les sélecteur en CSS)*
2. **Faire une opération**

4- Manipulation du DOM

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

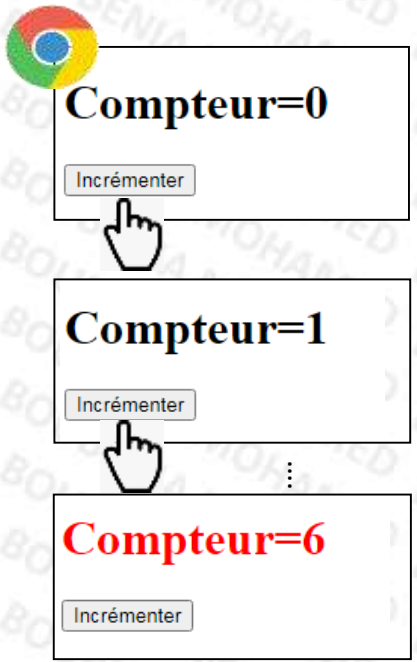
Dans cet exemple, nous avons d'abord retrouvé l'élément h1, avec son identifiant "compt_H1", et nous avons changé son contenu.

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Ici, nous avons changé son style CSS

Il existe plusieurs manières pour retrouver un élément HTML.

4- Manipulation du DOM

Retrouver un élément HTML

- Par identifiant
- Par le nom de balise
- Par class
- Par sélecteur CSS

Il existe plusieurs manières pour retrouver un élément HTML.

BOUBENIA MOHAMED

4- Manipulation du DOM

Retrouver un élément HTML

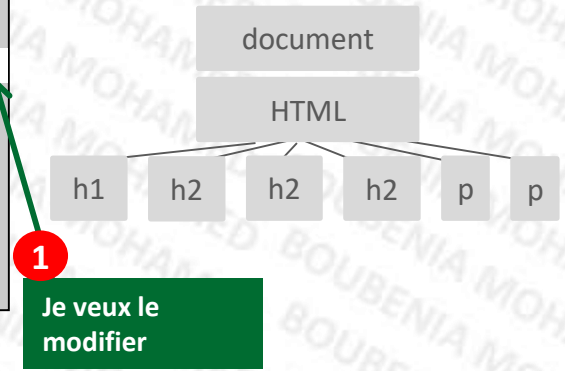
- **Par identifiant**
- **Par le nom de balise**
- **Par class**
- **Par sélecteur CSS**

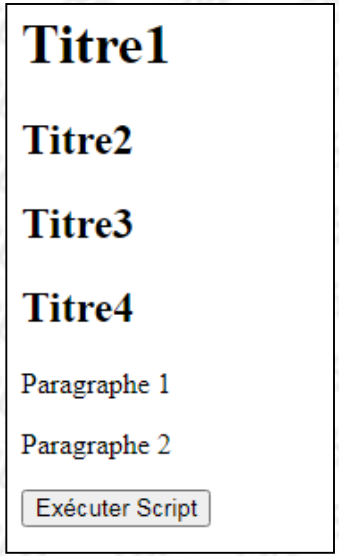
BOUBENIA MOHAMED

4- Manipulation du DOM

Retrouver un élément HTML — Par identifiant

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```





getElementById(<id>)

Retourne un seul élément qui possède l'identifiant id indiqué

```
function doTraitement(){
    // Je retrouve l'élément
    // et je le stock dans une variable elemnt
    element = document.getElementById("id_Titre2");
    element.innerHTML = "Titre2 modifié";

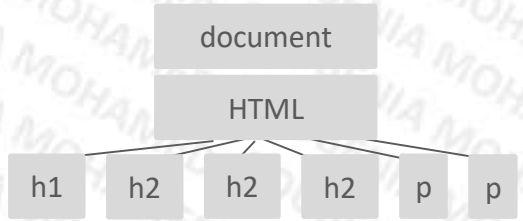
    // comme je peux faire les 2 à la fois
    //document.getElementById("id_Titre2").innerHTML = "Titre2 modifié";

}
```

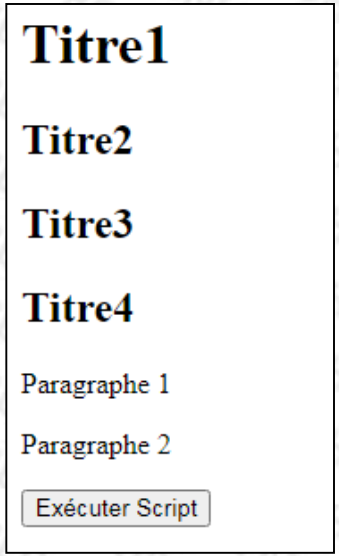
4- Manipulation du DOM

Retrouver un élément HTML — Par identifiant

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1. Je veux le modifier
2. Je peux le retrouver avec son id



getElementById(<id>)

Retourne un seul élément qui possède l'identifiant id indiqué

```
function doTraitement(){
    // Je retrouve l'élément
    // et je le stock dans une variable elemnt
    element = document.getElementById("id_Titre2");
    element.innerHTML = "Titre2 modifié";

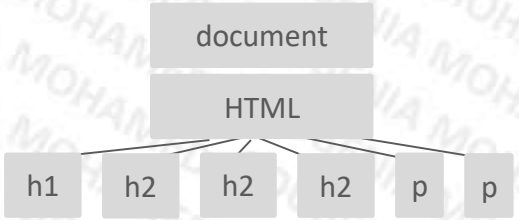
    // comme je peux faire les 2 à la fois
    //document.getElementById("id_Titre2").innerHTML = "Titre2 modifié";
}
```

3. On commence toujours par "document", car c'est lui la racine

4- Manipulation du DOM

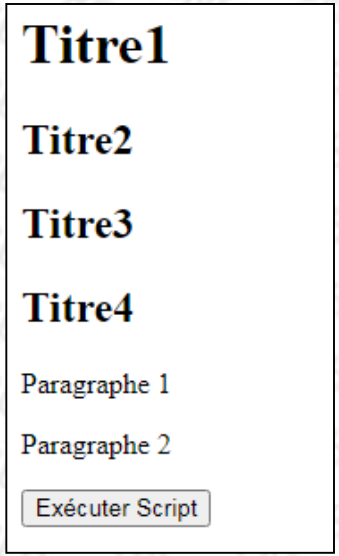
Retrouver un élément HTML — Par identifiant

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec son id



getElementById(<id>)

Retourne un seul élément qui possède l'identifiant id indiqué

```
function doTraitement(){
    // Je retrouve l'élément
    // et je le stock dans une variable elemnt
    element = document.getElementById("id_Titre2");
    element.innerHTML = "Titre2 modifié";

    // comme je peux faire les 2 à la fois
    //document.getElementById("id_Titre2").innerHTML = "Titre2 modifié";
}
```

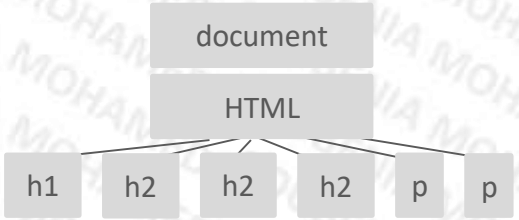
3 On commence toujours par "document", car c'est lui la racine

4 L'élément récupéré est un objet qui possède des propriété et des méthodes. "innerHTML" est une propriété qui renferme le contenu de l'élément. Donc la modifier, permettra d'avoir une différence au visuel dans la page Web.

4- Manipulation du DOM

Retrouver un élément HTML — Par identifiant

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec son id

getElementById(<id>)

Retourne un seul élément qui possède l'identifiant id indiqué

```
function doTraitement(){
    // Je retrouve l'élément
    // et je le stock dans une variable elemnt
    element = document.getElementById("id_Titre2");
    element.innerHTML = "Titre2 modifié";

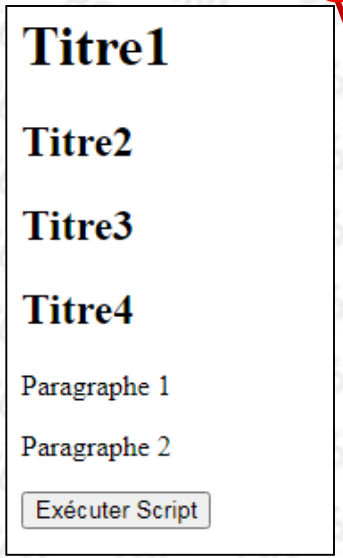
    // comme je peux faire les 2 à la fois
    //document.getElementById("id_Titre2").innerHTML = "Titre2 modifié";
}
```

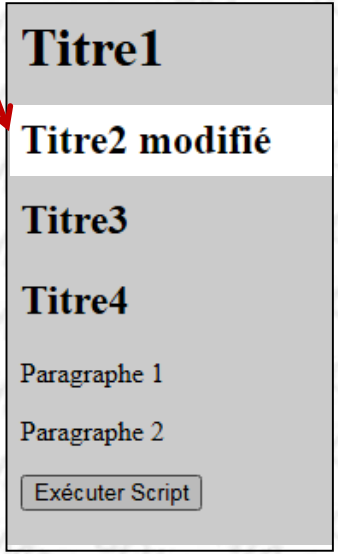
3 On commence toujours par "document", car c'est lui la racine

4 L'élément récupéré est un objet qui possède des propriété et des méthodes. "innerHTML" est une propriété qui renferme le contenu de l'élément. Donc la modifier, permettra d'avoir une différence au visuel dans la page Web.

L'élément touché par la modification Après clique sur bouton "Exécuter Script", qui déclenche la fonction doTraitement()

5





4- Manipulation du DOM

Retrouver un élément HTML

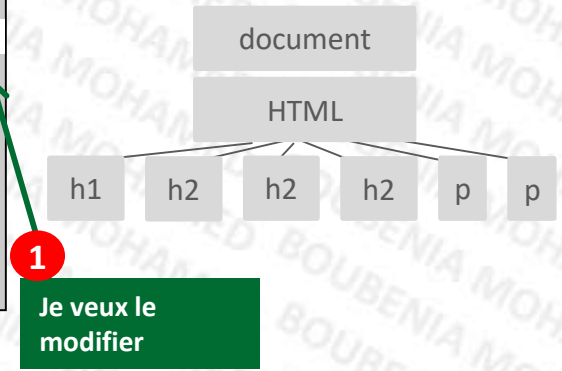
- Par identifiant
- **Par le nom de balise**
- Par class
- Par sélecteur CSS

BOUBENIA MOHAMED

4- Manipulation du DOM

Retrouver un élément HTML — Par le nom de balise

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



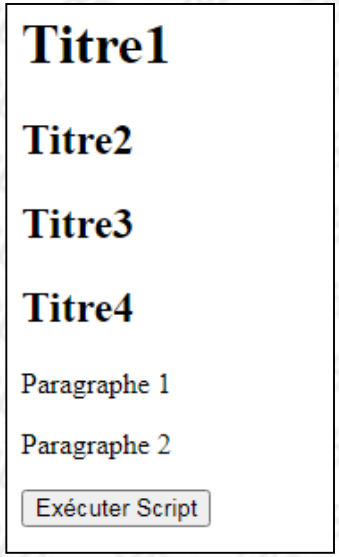
getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

```
function doTraitement(){

    // getElementsByTagName retourne une collection
    element = document.getElementsByTagName("h1")[0];
    element.innerHTML = "Titre1 modifié";

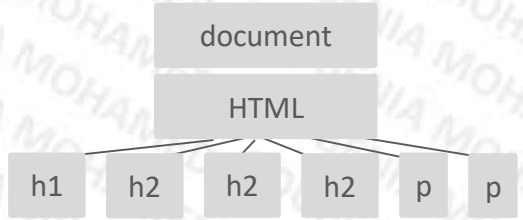
}
```



4- Manipulation du DOM

Retrouver un élément HTML — Par le nom de balise

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec le nom balise

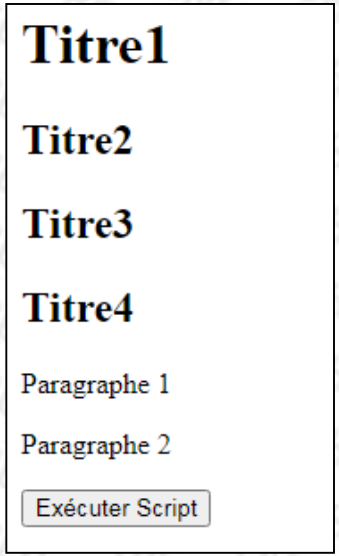
getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

```
function doTraitement(){

    // getElementsByTagName retourne une collection
    element = document.getElementsByTagName("h1")[0];
    element.innerHTML = "Titre1 modifié";

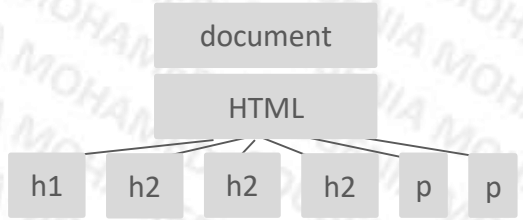
}
```



4- Manipulation du DOM

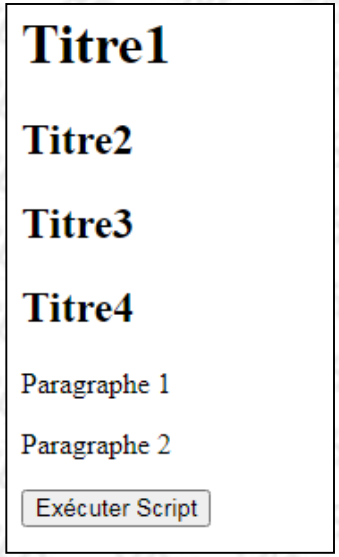
Retrouver un élément HTML — Par le nom de balise

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec le nom balise



getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

```
function doTraitement(){

    // getElementsByTagName retourne une collection
    element = document.getElementsByTagName("h1")[0];
    element.innerHTML = "Titre1 modifié";

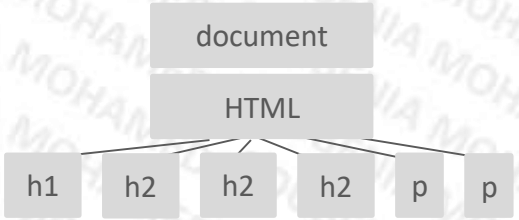
}
```

3 Même s'il y a un seul h1 dans notre page HTML, la méthode getElementsByTagName retourne toujours une collection.
Dans notre cas, la collection contiendra un seul élément h1, et pour le retrouver, on lit le premier item de cet collection, avec [0] (car une collection commence de l'index 0)

4- Manipulation du DOM

Retrouver un élément HTML — Par le nom de balise

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec le nom balise

getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

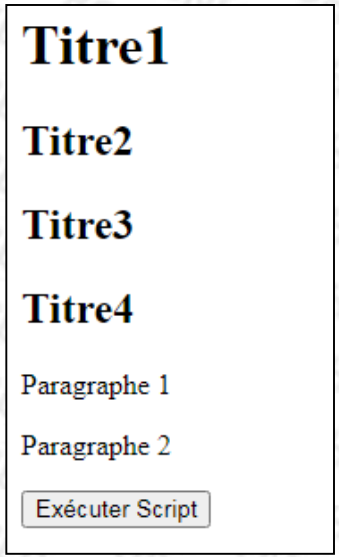
```
function doTraitement(){

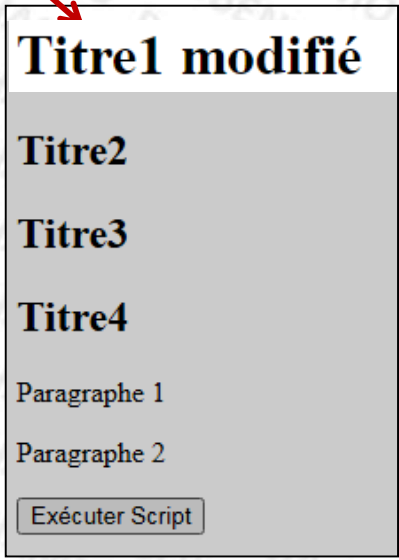
    // getElementsByTagName retourne une collection
    element = document.getElementsByTagName("h1")[0];
    element.innerHTML = "Titre1 modifié";

}
```

L'élément touché par la modification Après clique sur bouton "Exécuter Script", qui déclenche la fonction doTraitement()

4





3 Même s'il y a un seul h1 dans notre page HTML, la méthode getElementsByTagName retourne toujours une collection.
Dans notre cas, la collection contiendra un seul élément h1, et pour le retrouver, on lit le premier item de cet collection, avec [0] (car une collection commence de l'index 0)

4- Manipulation du DOM

Retrouver un élément HTML

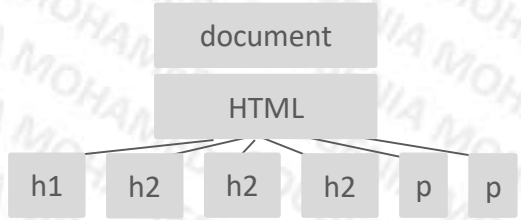
- Par identifiant
- Par le nom de balise
- **Par class**
- Par sélecteur CSS

BOUBENIA MOHAMED

4- Manipulation du DOM

Retrouver un élément HTML — Par class

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

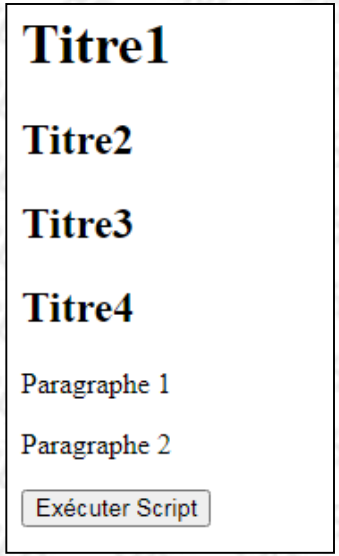
getElementsByClassName(<class>)

Retourne toujours une collection d'éléments d'une classe données

```
function doTraitement(){

    // getElementsByClassName retourne une collection
    element = document.getElementsByClassName("class_p2")[0];
    element.innerHTML = "Paragraphe 2 modifié";

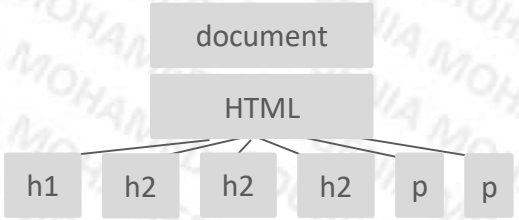
}
```



4- Manipulation du DOM

Retrouver un élément HTML — Par class

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec le nom classe

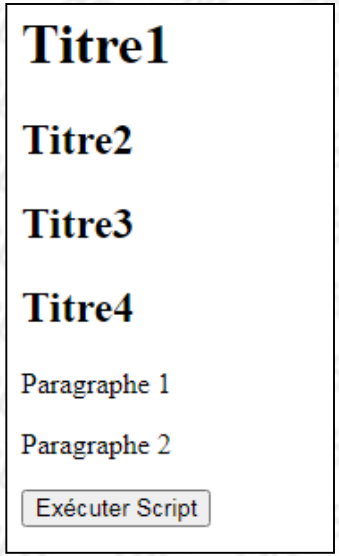
getElementsByClassName(<class>)

Retourne toujours une collection d'éléments d'une classe données

```
function doTraitement(){

    // getElementsByClassName retourne une collection
    element = document.getElementsByClassName("class_p2")[0];
    element.innerHTML = "Paragraphe 2 modifié";

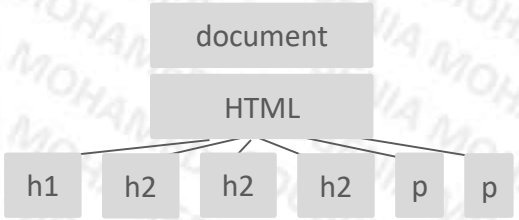
}
```



4- Manipulation du DOM

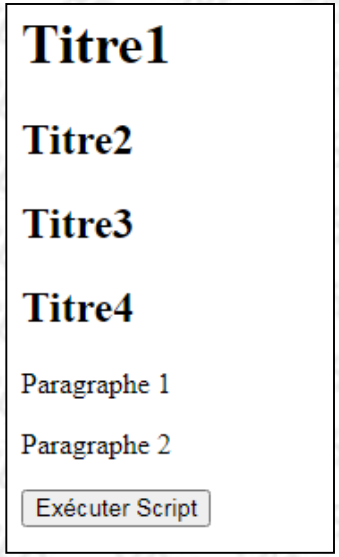
Retrouver un élément HTML — Par class

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec le nom classe



getElementsByClassName(<class>)

Retourne toujours une collection d'éléments d'une classe données

```
function doTraitement(){

    // getElementsByClassName retourne une collection
    element = document.getElementsByClassName("class_p2")[0];
    element.innerHTML = "Paragraphe 2 modifié";

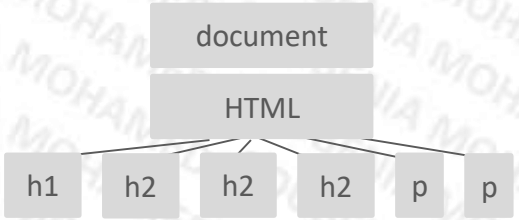
}
```

3 Dans notre cas, la collection contiendra un seul élément p, et pour le retrouver, on lit le premier item de cet collection, avec [0] (car une collection commence de l'index 0)

4- Manipulation du DOM

Retrouver un élément HTML — Par class

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



1 Je veux le modifier

2 Je peux le retrouver avec le nom classe

getElementsByClassName(<class>)

Retourne toujours une collection d'éléments d'une classe données

```
function doTraitement(){

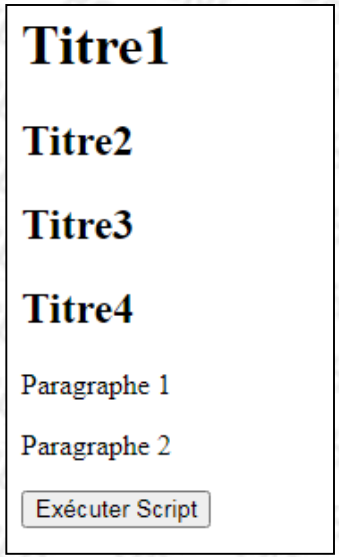
    // getElementsByClassName retourne une collection
    element = document.getElementsByClassName("class_p2")[0];
    element.innerHTML = "Paragraphe 2 modifié";

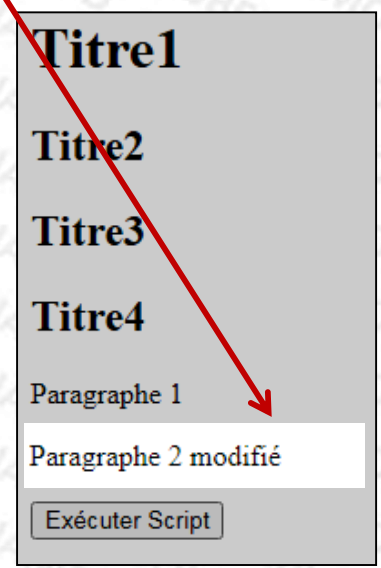
}
```

3 Dans notre cas, la collection contiendra un seul élément p, et pour le retrouver, on lit le premier item de cet collection, avec [0] (car une collection commence de l'index 0)

L'élément touché par la modification Après clique sur bouton "Exécuter Script", qui déclenche la fonction doTraitement()

4





4- Manipulation du DOM

Retrouver un élément HTML

- Par identifiant
- Par le nom de balise
- Par class
- Par sélecteur CSS

BOUBENIA MOHAMED

4- Manipulation du DOM

Retrouver un élément HTML — Par sélecteur CSS

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <div>
        <p>Paragraphe 3</p>
    </div>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux le modifier

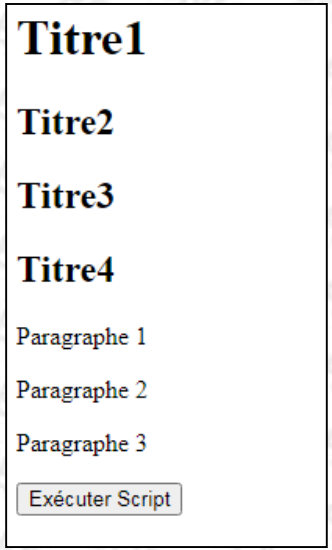
querySelectorAll(<sélecteur>)

Retourne toujours une collection d'éléments d'un sélecteur donné

```
function doTraitement(){

    // querySelectorAll retourne une collection
    element = document.querySelectorAll("div p")[0];
    element.innerHTML = "Paragraphe 3 modifié";

}
```



4- Manipulation du DOM

Retrouver un élément HTML — Par sélecteur CSS

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <div>
        <p>Paragraphe 3</p>
    </div>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux le modifier

2 Je peux le retrouver avec un Sélecteur CSS

querySelectorAll(<sélecteur>)

Retourne toujours une collection d'éléments d'un sélecteur donné

```
function doTraitement(){

    // querySelectorAll retourne une collection
    element = document.querySelectorAll("div p")[0];
    element.innerHTML = "Paragraphe 3 modifié";

}
```

Titre1

Titre2

Titre3

Titre4

Paragraphe 1

Paragraphe 2

Paragraphe 3

Exécuter Script

4- Manipulation du DOM

Retrouver un élément HTML — Par sélecteur CSS

```
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <div>
        <p>Paragraphe 3</p>
    </div>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux le modifier

2 Je peux le retrouver avec un Sélecteur CSS

querySelectorAll(<sélecteur>)

Retourne toujours une collection d'éléments d'un sélecteur donné

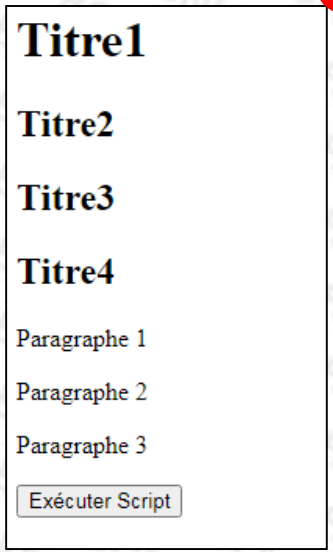
```
function doTraitement(){

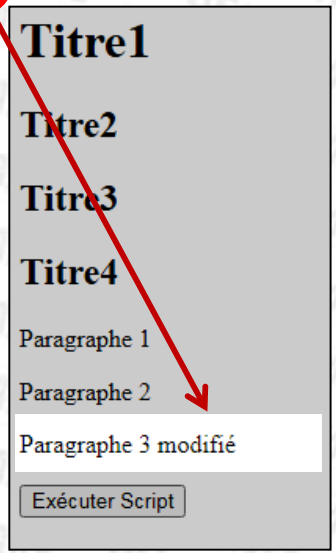
    // querySelectorAll retourne une collection
    element = document.querySelectorAll("div p")[0];
    element.innerHTML = "Paragraphe 3 modifié";

}
```

L'élément touché par la modification Après clique sur bouton "Exécuter Script", qui déclenche la fonction doTraitement()

3





4- Manipulation du DOM

Retrouver plusieurs éléments HTML

- Par identifiant
- Par le nom de balise
- Par class
- Par sélecteur CSS

Jusqu'à présent, on n'a vu que l'exemple avec un seul élément récupéré…

….Mais, imaginons que ma sélection retourne plusieurs éléments

BOUBENIA MOHAMED

4- Manipulation du DOM

Retrouver plusieurs éléments HTML

```html
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux les modifier

getElementsByTagName(<tag>)

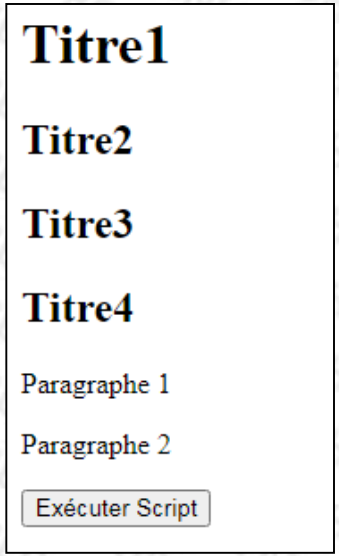
Retourne toujours une collection d'éléments d'une balise données

```javascript
function doTraitement(){

    // va retourner une collection de plusieurs éléments h2
    elements_h2 = document.getElementsByTagName("h2");

    // Utiliser une boucle pour parcourir plusieur éléments sélectionnés
    for (let i = 0; i < elements_h2.length; i++) {
        elements_h2[i].innerHTML = elements_h2[i].innerHTML + " modifié";
    }

}
```



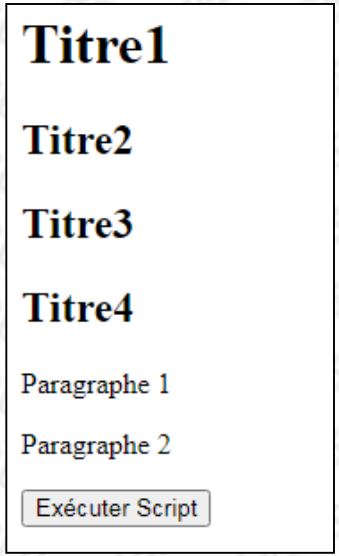
4- Manipulation du DOM

Retrouver plusieurs éléments HTML

```html
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux les modifier

2 Je peux les retrouver avec le nom Balise h2



getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

```javascript
function doTraitement(){

    // va retourner une collection de plusieurs éléments h2
    elements_h2 = document.getElementsByTagName("h2");

    // Utiliser une boucle pour parcourir plusieur éléments sélectionnés
    for (let i = 0; i < elements_h2.length; i++) {
        elements_h2[i].innerHTML = elements_h2[i].innerHTML + " modifié";
    }

}
```

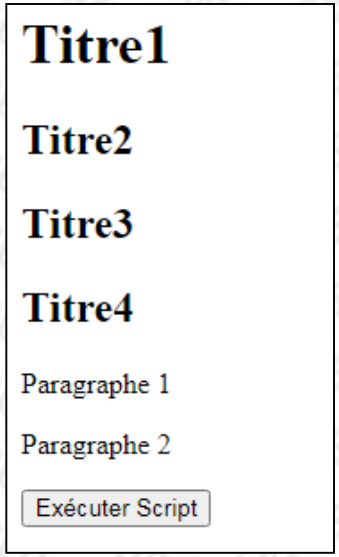
4- Manipulation du DOM

Retrouver plusieurs éléments HTML

```html
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux les modifier

2 Je peux les retrouver avec le nom Balise h2



getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

```
function doTraitement(){

    // va retourner une collection de plusieurs éléments h2
    elements_h2 = document.getElementsByTagName("h2");

    // Utiliser une boucle pour parcourir plusieur éléments sélectionnés
    for (let i = 0; i < elements_h2.length; i++) {
        elements_h2[i].innerHTML = elements_h2[i].innerHTML + " modifié";
    }

}
```

3 Comme getElementsByTagName retourne toujours une collection, et ma collection contient les 3 élément h2 présent dans mon document HTML, je dois faire une boucle pour parcourir la sélection, et faire mon traitement.

4- Manipulation du DOM

Retrouver plusieurs éléments HTML

```html
<body>
    <h1>Titre1</h1>
    <h2 id="id_Titre2">Titre2</h2>
    <h2>Titre3</h2>
    <h2>Titre4</h2>
    <p>Paragraphe 1</p>
    <p class="class_p2">Paragraphe 2</p>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

1 Je veux les modifier

2 Je peux les retrouver avec le nom Balise h2

getElementsByTagName(<tag>)

Retourne toujours une collection d'éléments d'une balise données

```js
function doTraitement(){

    // va retourner une collection de plusieurs éléments h2
    elements_h2 = document.getElementsByTagName("h2");

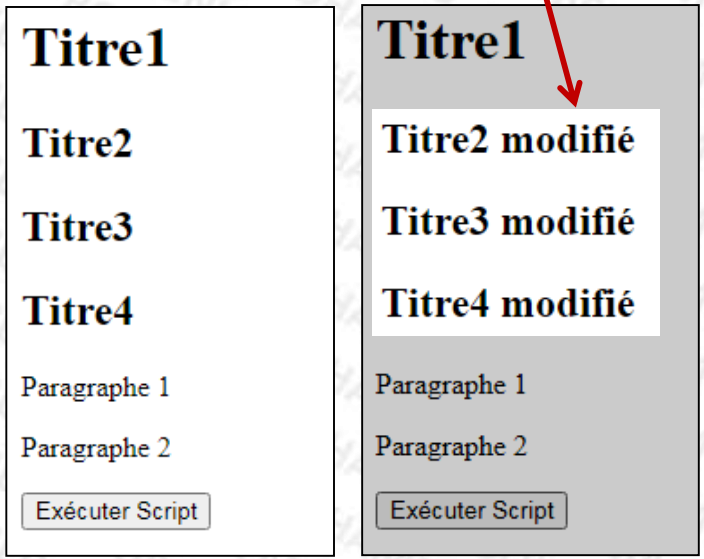
    // Utiliser une boucle pour parcourir plusieur éléments sélectionnés
    for (let i = 0; i < elements_h2.length; i++) {
        elements_h2[i].innerHTML = elements_h2[i].innerHTML + " modifié";
    }

}
```

3 Comme getElementsByTagName retourne toujours une collection, et ma collection contient les 3 élément h2 présent dans mon document HTML, je dois faire une boucle pour parcourir la sélection, et faire mon traitement.

L'élément touché par la modification Après clique sur bouton "Exécuter Script", qui déclenche la fonction doTraitement()

4



4- Manipulation du DOM

Retrouver un élément HTML

- Par identifiant
- Par le nom de balise
- Par class
- Par sélecteur CSS

1- Retrouver l'élément HTML



BOUBENIA MOHAMED

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1. **Retrouver l'élément HTML**
2. **Faire une opération**

Une fois l'élément retrouvé, on peut faire des opérations dessus

Un élément HTML

BOUBENIA MOHAMED

4- Manipulation du DOM

D'abord, il faut savoir qu'en JavaScript, un élément HTML est un objet qui possède des propriétés et des méthodes

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML



2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Un élément HTML

Propriétés

- parentNode
- children
- firstElementChild
- lastElementChild
- nextElementSibling
- innerHTML
- innerText
- attributs
- style

Méthodes

- append()
- remove()
- removeChild()
- appendChild()
- replaceChild()
- hasAttribute()
- getAttribute()
- setAttribute()

Les plus importantes

Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

D'abord, il faut savoir qu'en JavaScript, un élément HTML est un objet qui possède des propriétés et des méthodes

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML



2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes

Un élément HTML

Propriétés

- parentNode
- children
- firstElementChild
- lastElementChild
- nextElementSibling
- innerHTML
- innerText
- attributs
- style

Méthodes

- append()
- remove()
- removeChild()
- replaceChild()
- appendChild()
- hasAttribute()
- getAttribute()
- setAttribute()

Les plus importantes

Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

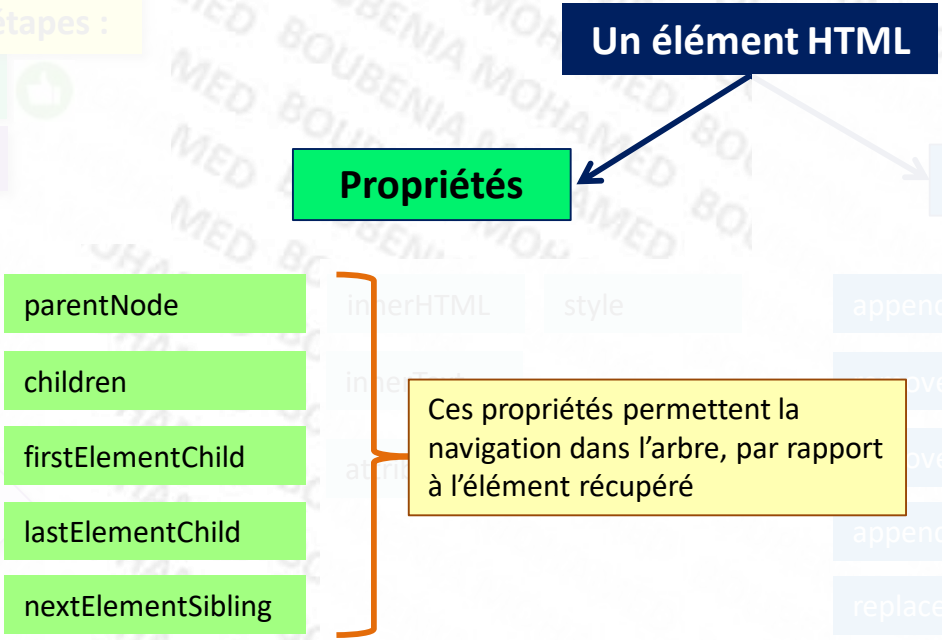
Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes



innerHTML style

append() hasAttribute()

getAttribute()

setAttribute()

appendChild()

replaceChild()

Les plus importantes

Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

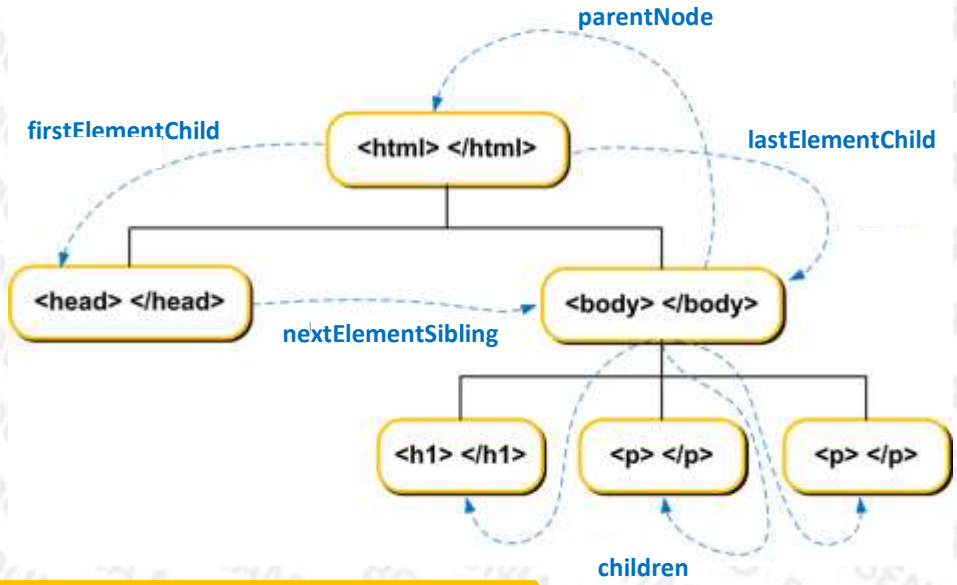
4- Manipulation du DOM

Un élément HTML

Propriétés

- parentNode
- children
- firstElementChild
- lastElementChild
- nextElementSibling

Ces propriétés permettent la navigation dans l'arbre, par rapport à l'élément récupéré



Exemple:
Si l'élément récupéré c'est body.
La propriété children de cet élément donnera la collection d'élément h1, p, p

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération



parentNode	innerHTML	style	append()
children	innerText		
firstElementChild	attributs		
lastElementChild			appendChild()
nextElementSibling			replaceChild()

Permettent la modification du contenu de l'élément.

Les plus importantes

Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

Un élément HTML

Propriétés

- innerHTML
- innerText

Permettent la modification du contenu de l'élément.

```
<body>

</body>
```

```
element = document.getElementsByTagName("body")[0];
```

```
element.innerHTML = "Text inséré avec InnerHTML";
```

Text inséré avec InnerHTML

innerHTML est reconnue comme du code HTML

```
element.innerHTML = "<h1>Element H1 inséré avec innerHTML</h1>";
```

Element H1 inséré avec innerHTML

```
element.innerText = "Text inséré avec innerText";
```

Text inséré avec innerText

innerText est reconnue comme du texte simple

```
element.innerText = "<h1>Element H1 inséré avec InnerText</h1>";
```

<h1>Element H1 inséré avec InnerText</h1>

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes

Un élément HTML

Propriétés

Méthodes

parentNode	innerHTML	style
childNodes	innerText	
firstChild	attributs	
lastChild		
nextSibling		

Il est possible d'accéder à certains attributs directement avec leur nom

append()	hasAttribute()
remove()	getAttribute()
	setAttribute()
replaceChild()	

Les plus importantes

Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

Un élément HTML

Propriétés

attributs

Il est possible d'accéder à certains attributs directement avec leur nom

HTML

```
<body>
    <a href="http://www.usthb.dz">Mon Lien</a>
    <img src="monImage.png" />
    <h1 style="color:red">Titre 1</h1>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>

<script>
    function doTraitement(){

        element_a = document.getElementsByTagName("a")[0];
        element_img = document.getElementsByTagName("img")[0];
        element_h1 = document.getElementsByTagName("h1")[0];

        console.log(element_a.href);
        console.log(element_img.src);
        console.log(element_h1.style);
    }
</script>
```

Console

```
http://www.usthb.dz/
file:///D:/MOHAMED USTHB/2021-2022/M1 PRAVAN mind/BOUBENIA/monImage.png
▶ CSSStyleDeclaration {0: 'color', accentColor: '', additiveSymbols: '', alignContent: '', alignItems: '', alignSelf: '', …}
>
```

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes



parentNode	innerHTML	style
childNodes	innerText	
firstChild	attributs	
lastChild		
nextSibling		

Il est possible de modifier le style CSS d'un élément directement à travers la propriété style.

removeChild() setAttribute()

appendChild()

replaceChild()

Les plus importantes

Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

Un élément HTML

Propriétés

style

Il est possible de modifier le style CSS d'un élément directement à travers la propriété style.

Titre 1

```
<body>
    <h1>Titre 1</h1>
</body>
```

```
element_h1 = document.getElementsByTagName("h1")[0];
```

Titre 1

```
element_h1.style.color="red";
```

Titre 1

```
element_h1.style.backgroundColor="red";
```

Titre 1

```
element_h1.style.borderTop="2px solid black";
```

Remarque: en JavaScript, les tirets (-) dans les propriétés CSS sont supprimé, et remplacés par la première lettre en majuscule du mot suivant

CSS	Background-color
JS	BackgroundColor

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes

Un élément HTML

Propriétés

parentNode	innerHTML	style
childNodes	innerText	
firstChild	attributs	
lastChild		
nextSibling		

Méthodes

append()	hasAttribute()
remove()	getAttribute()
removeChild()	setAttribute()
replaceChild()	
appendChild()	

Les plus importantes

ir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes

Un élément HTML

Propriétés

Méthodes

parentNode

childNodes

firstChild

lastChild

nextSibling

attributs

Rajoute du texte au contenu existant, sans le remplacer.

append()

remove()

removeChild()

appendChild()

replaceChild()

hasAttribute()

getAttribute()

setAttribute()

Les plus importantes

ir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

Un élément HTML

Méthodes

append()

```
<body>
    <h1>Titre 1</h1>
</body>
```

Titre 1

```
element_h1 = document.getElementsByTagName("h1")[0];
```

```
element_h1.innerHTML = element_h1.innerHTML + " modifié";
```

Titre 1 modifié

Manière plus simple de rajouter à la fin

```
element_h1.append("modifié");
```

Titre 1 modifié

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes

Un élément HTML

Propriétés

Méthodes

parentNode | innerHTML | style

childNodes | innerText

firstChild

lastChild

nextSibling

Permettent de supprimer/ rajouter/remplacer un élément

- append()
- remove()
- removeChild()
- replaceChild()
- appendChild()

hasAttribute()

getAttribute()

setAttribute()

Les plus importantes

ir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

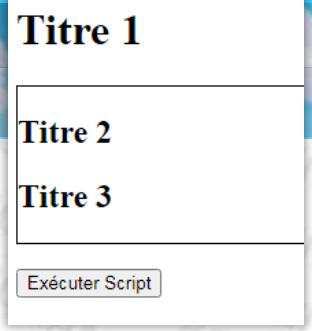
Un élément HTML

Méthodes

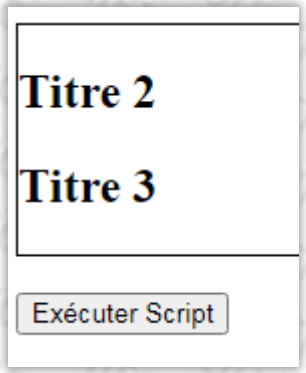
- remove()
- removeChild()
- replaceChild()
- appendChild()

Permet de supprimer l'élément

```
<body>
    <h1>Titre 1</h1>
    <div style="border:1px solid black">
        <h2>Titre 2</h2>
        <h2>Titre 3</h2>
    </div>
    </br>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```




```
element_h1 = document.getElementsByTagName("h1")[0];
element_h1.remove();
```



L'élément Titre 1 a été supprimé.

4- Manipulation du DOM

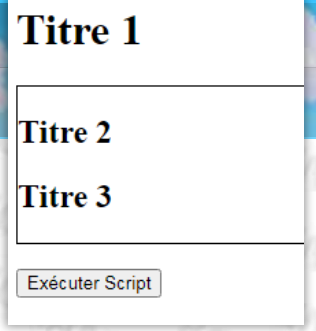
Un élément HTML

Méthodes

- remove()
- **removeChild()**
- replaceChild()
- appendChild()

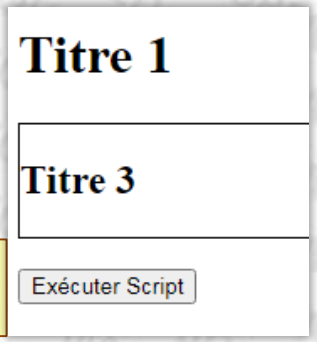
Permet de supprimer un élément enfant

```
<body>
    <h1>Titre 1</h1>
    <div style="border:1px solid black">
        <h2>Titre 2</h2>
        <h2>Titre 3</h2>
    </div>
    </br>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

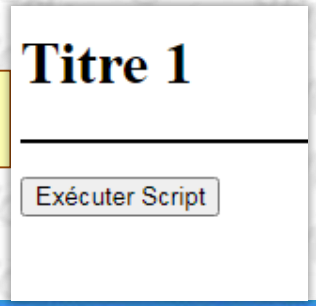



```
// sélectionner le div
element_div = document.getElementsByTagName("div")[0];
// sélectionner un élément enfant h2 avec un sélecteur CSS ">"
element_h2  = document.querySelectorAll("div>h2")[0];
// supprimer l'élément enfant du div
element_div.removeChild(element_h2);
```

Au premier clic, Titre2 est supprimé.



Au deuxième clic, Titre3 est supprimé.



Au troisième clic, on obtient une erreur, car plus aucun h2 ne se trouve dans le div

4- Manipulation du DOM

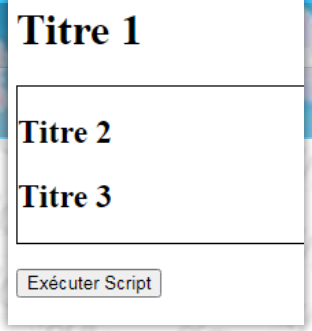
Un élément HTML

Méthodes

- remove()
- removeChild()
- **replaceChild()**
- appendChild()

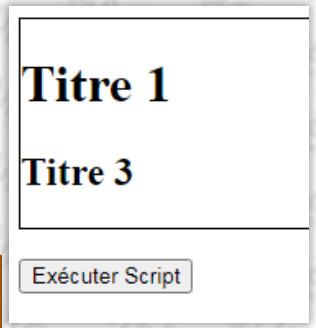
Permet de remplacer un élément enfant

```
<body>
    <h1>Titre 1</h1>
    <div style="border:1px solid black">
        <h2>Titre 2</h2>
        <h2>Titre 3</h2>
    </div>
    </br>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```




```
// sélectionner l'élément h1
element_h1  = document.getElementsByTagName("h1")[0];
// sélectionner l'élément h2 du div
element_h2  = document.querySelectorAll("div>h2")[0];
// sélectionner le parent de h2
element_div = document.getElementsByTagName("div")[0];
// remplacer element_h2 par l'element_h1
element_div.replaceChild(element_h1, element_h2);
```

Au premier clic, Titre 2 est remplacé par Titre 1 (Titre 1 a changé de place dans le DOM)



Au deuxième clic, Titre 3 est remplacé par Titre 1 encore



Au troisième clic, on obtient une erreur, car plus aucun h2 ne se trouve dans le div

4- Manipulation du DOM

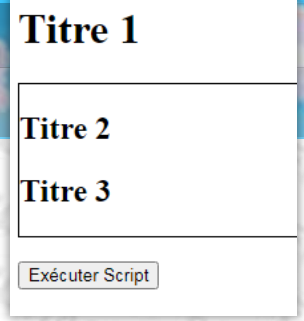
Un élément HTML

Méthodes

- remove()
- removeChild()
- replaceChild()
- **appendChild()**

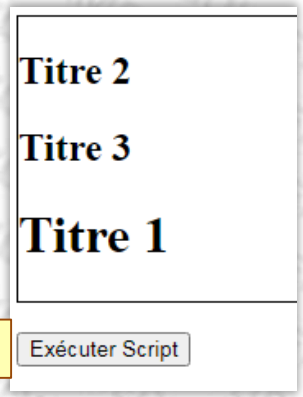
Permet de rajouter un élément enfant à la fin

```
<body>
    <h1>Titre 1</h1>
    <div style="border:1px solid black">
        <h2>Titre 2</h2>
        <h2>Titre 3</h2>
    </div>
    </br>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```




```
// sélectionner l'élément h1
element_h1  = document.getElementsByTagName("h1")[0];
// sélectionner l'élément div
element_div = document.getElementsByTagName("div")[0];
// rajouter element_h1 au div (à la fin)
element_div.appendChild(element_h1);
```

Titre 1 (Titre 1 a changé de place dans le DOM)



4- Manipulation du DOM

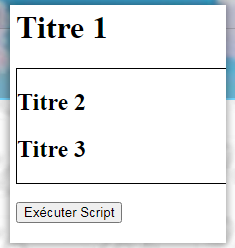
Un élément HTML

Méthodes

- remove()
- removeChild()
- replaceChild()
- **appendChild()**

Permet de rajouter un élément enfant

```
<body>
    <h1>Titre 1</h1>
    <div style="border:1px solid black">
        <h2>Titre 2</h2>
        <h2>Titre 3</h2>
    </div>
    </br>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



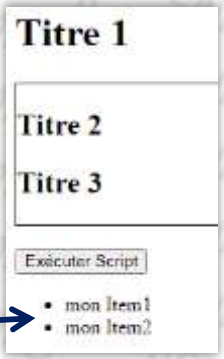
Je peux créer un élément HTML à partir de 0, et l'insérer dans le DOM

```
// créer un élément ul vide
list = document.createElement("ul");
// créer un élément li vide
list_item1 = document.createElement("li");
// remplir le contenu de list_item1
list_item1.innerText= "mon Item1";

// créer un élément li vide
list_item2 = document.createElement("li");
// remplir le contenu de list_item1
list_item2.innerText= "mon Item2";

// rajouter list_item1 & list_item2 comme enfant de list
list.appendChild(list_item1);
list.appendChild(list_item2);

// rajouter list comment enfant à body, pour le rendre visible
document.getElementsByTagName("body")[0].appendChild(list);
// ou simplement
//document.body.appendChild(list);
```



4- Manipulation du DOM

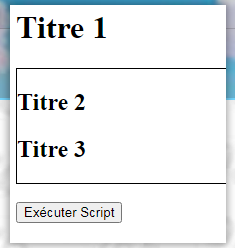
Un élément HTML

Méthodes

- remove()
- removeChild()
- replaceChild()
- appendChild()

Permet de rajouter un élément enfant

```
<body>
    <h1>Titre 1</h1>
    <div style="border:1px solid black">
        <h2>Titre 2</h2>
        <h2>Titre 3</h2>
    </div>
    </br>
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```



Je peux créer un élément HTML à partir de 0, et l'insérer dans le DOM

```
// créer un élément ul vide
list = document.createElement("ul");
// créer un élément li vide
list_item
// rempli
list_item

// créer
list_item
// rempli
list_item

// rajout                                    list
list.appe
list.appe

// rajouter list comment enfant à body, pour le rendre visible
document.getElementsByTagName("body")[0].appendChild(list);
// ou simplement
//document.body.appendChild(list);
```

Une autre possibilité (mais moins élégante)

```
// déclaration textuel
element = "<ul>"
        + " <li>mon Item1</li>"
        + " <li>mon Item2</li>"
        + "</ul>";

// rajouter à la fin de body
document.body.innerHTML=
    document.body.innerHTML+element;
```



4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1- Retrouver l'élément HTML

2- Faire une opération

Une fois l'élément retrouvé, on peut faire des opérations dessus

Nos opérations vont, donc, soit, modifier les valeurs de ses propriétés, ou appeler une des ses méthodes



Propriétés

parentNode | innerHTML | style
childNodes | innerText
firstChild | attributs
lastChild
nextSibling

Permettent de d'accéder aux attributs de l'élément

- append()
- removeChild()
- replaceChild()
- appendChild()
- hasAttribute()
- getAttribute()
- setAttribute()

Les plus importantes

ir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

4- Manipulation du DOM

```html
<body>
    <img src="monImage.png" />
    <button onclick="doTraitement()">
        Exécuter Script
    </button>
</body>
```

Un élément HTML

Méthodes

- hasAttribute()
- getAttribute()
- setAttribute()

Permettent de d'accéder aux attributs de l'élément

```js
// récupérer l'élément img
element_a = document.getElementsByTagName("img")[0];

// hasAttribute vérifie si l'attribut spécifié possède une valeur
console.log(element_a.hasAttribute("src")); // true
console.log(element_a.hasAttribute("alt")); // false

// getAttribute retourne la valeur de l'arribut spécifié
console.log(element_a.getAttribute("src"));
console.log(element_a.src);

// setAttribute modifie la valeur de l'arribut spécifié
element_a.setAttribute("src", "autreImage.png");
console.log(element_a.getAttribute("src"));
console.log(element_a.src);
```

```
true
false
monImage.png
file:///D:/MOHAMED_USTHB/2021-2022/M1_PRAVAN_mind/BOUBENIA/monImage.png
autreImage.png
file:///D:/MOHAMED_USTHB/2021-2022/M1_PRAVAN_mind/BOUBENIA/autreImage.png
```

Remarquez la différence entre:
getAttribute("src")
et **element.src**

getAttribute retourne la valeur exacte de l'attribut !

4- Manipulation du DOM

Un élément HTML

Méthodes

- classList.add()
- classList.remove()
- classList.toggle()

```
<button onclick="ajouterClass()">Add</button>
<button onclick="supprimerClass()">Delete</button>
<button onclick="switchClass()">Toggle</button>
<div id="myDIV">
    <p>I am a myDIV.</p>
</div>
```

```
.myStyle {
  background-color: coral;
  padding: 16px;
}
```

```
function ajouterClass() {
  const list = document.getElementById("myDIV").classList;
  list.add("myStyle");
}
```

```
function supprimerClass() {
  const list = document.getElementById("myDIV").classList;
  list.remove("myStyle");
}
```

```
function switchClass() {
  const list = document.getElementById("myDIV").classList;
  list.toggle("myStyle");
}
```



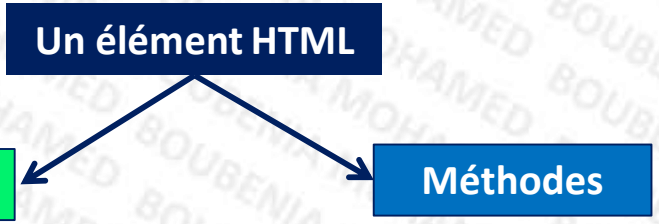


Toggle permet d'ajouter (ou supprimer) la classe si elle n'existe pas (ou existe).
Note: la propriété classListe donne la collection de classe affectée à l'élément.

4- Manipulation du DOM

Le tout se fait généralement en 2 étapes :

1. 1- Retrouver l'élément HTML
2. 2- Faire une opération



parentNode	innerHTML	style
childNodes	innerText	
firstChild	attributs	
lastChild		
nextSibling		

append()	hasAttribute()
remove()	getAttribute()
removeChild()	setAttribute()
replaceChild()	
appendChild()	

Les plus importantes

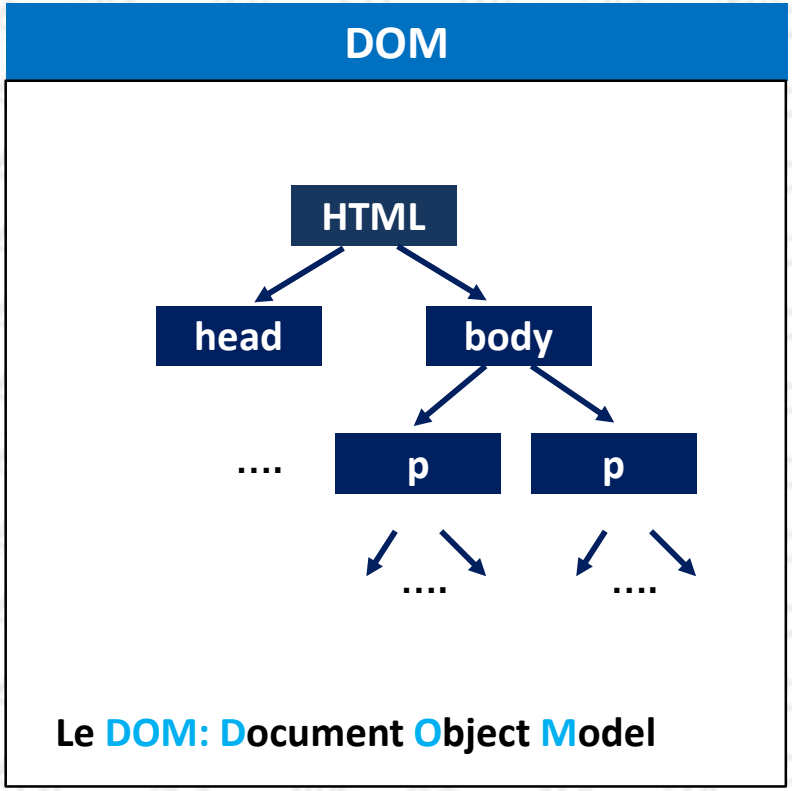
Voir d'autres ici: https://www.w3schools.com/jsref/dom_obj_all.asp

DOM *vs* BOM

BOUBENIA MOHAMED

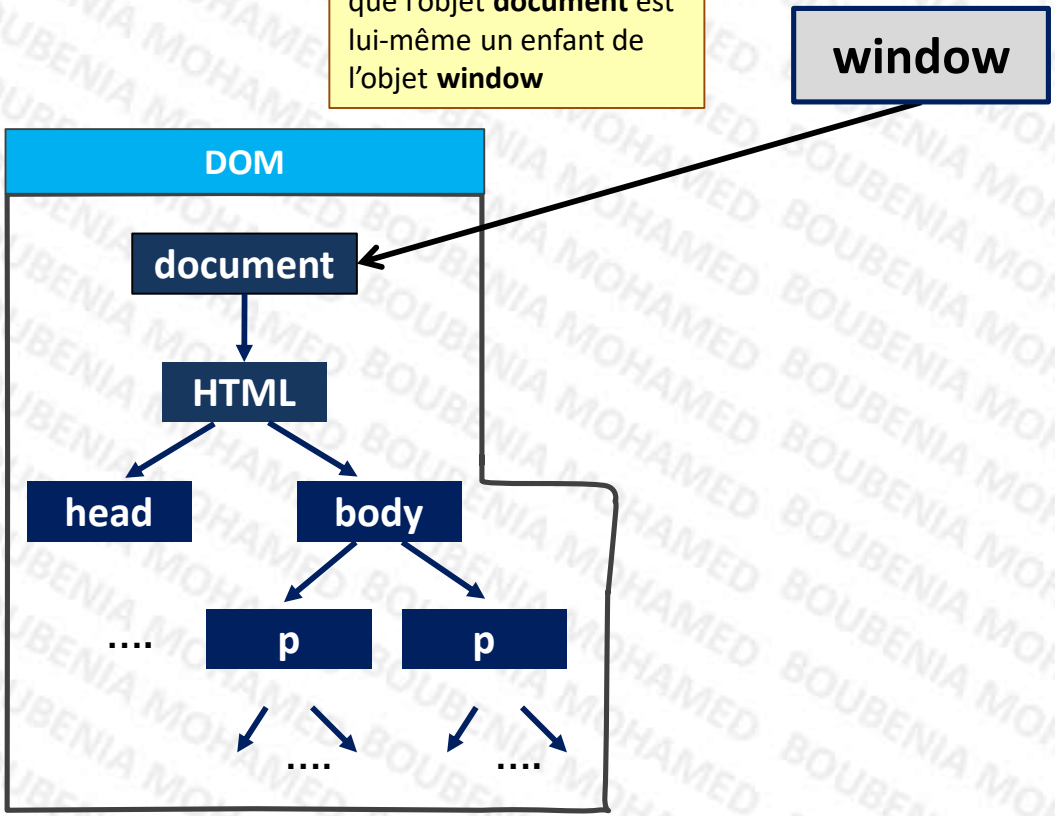
5- Le BOM (Browser Object Model)

Rappelez-vous



5- Le BOM (Browser Object Model)

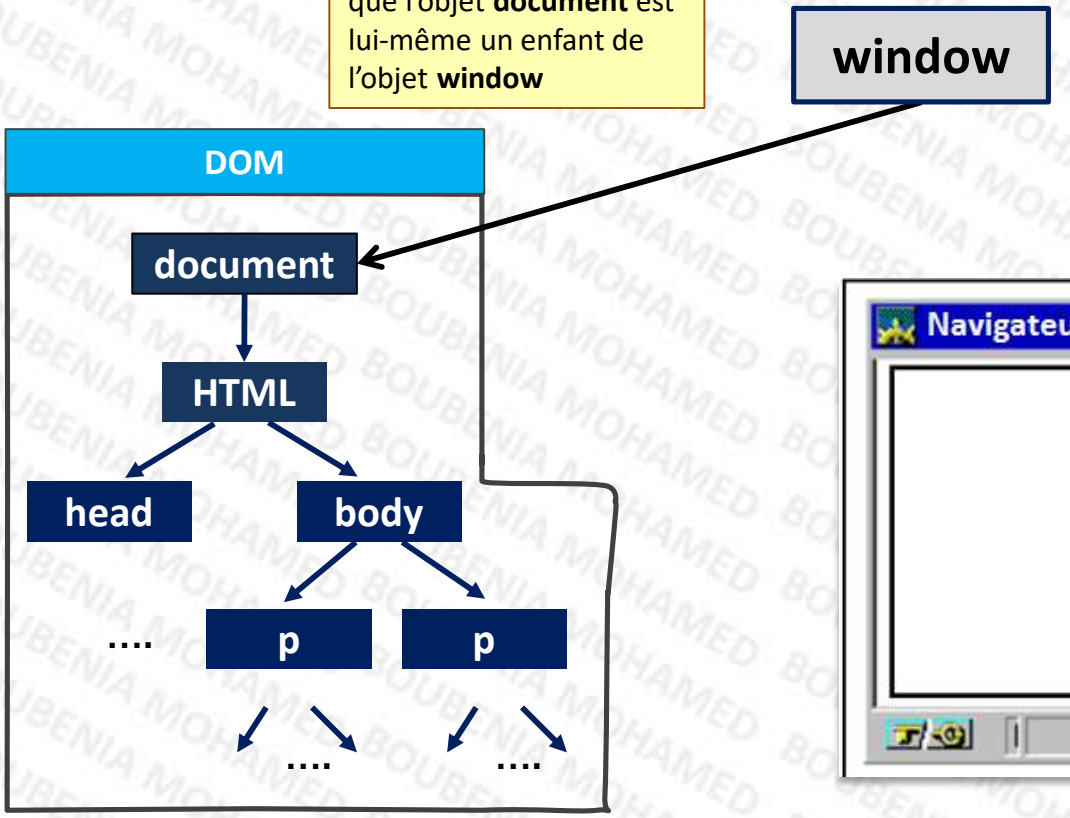
Maintenant, il faut savoir que l'objet **document** est lui-même un enfant de l'objet **window**

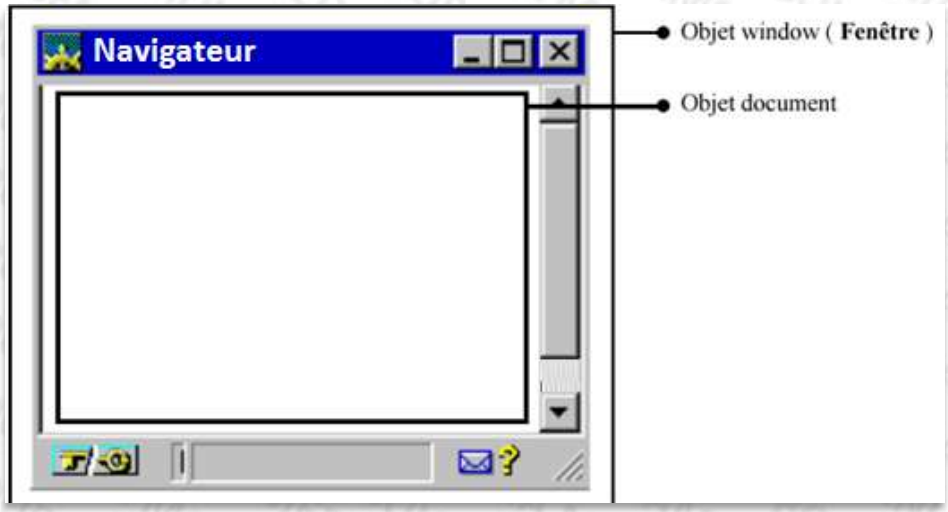


BOUBENIA MOHAMED

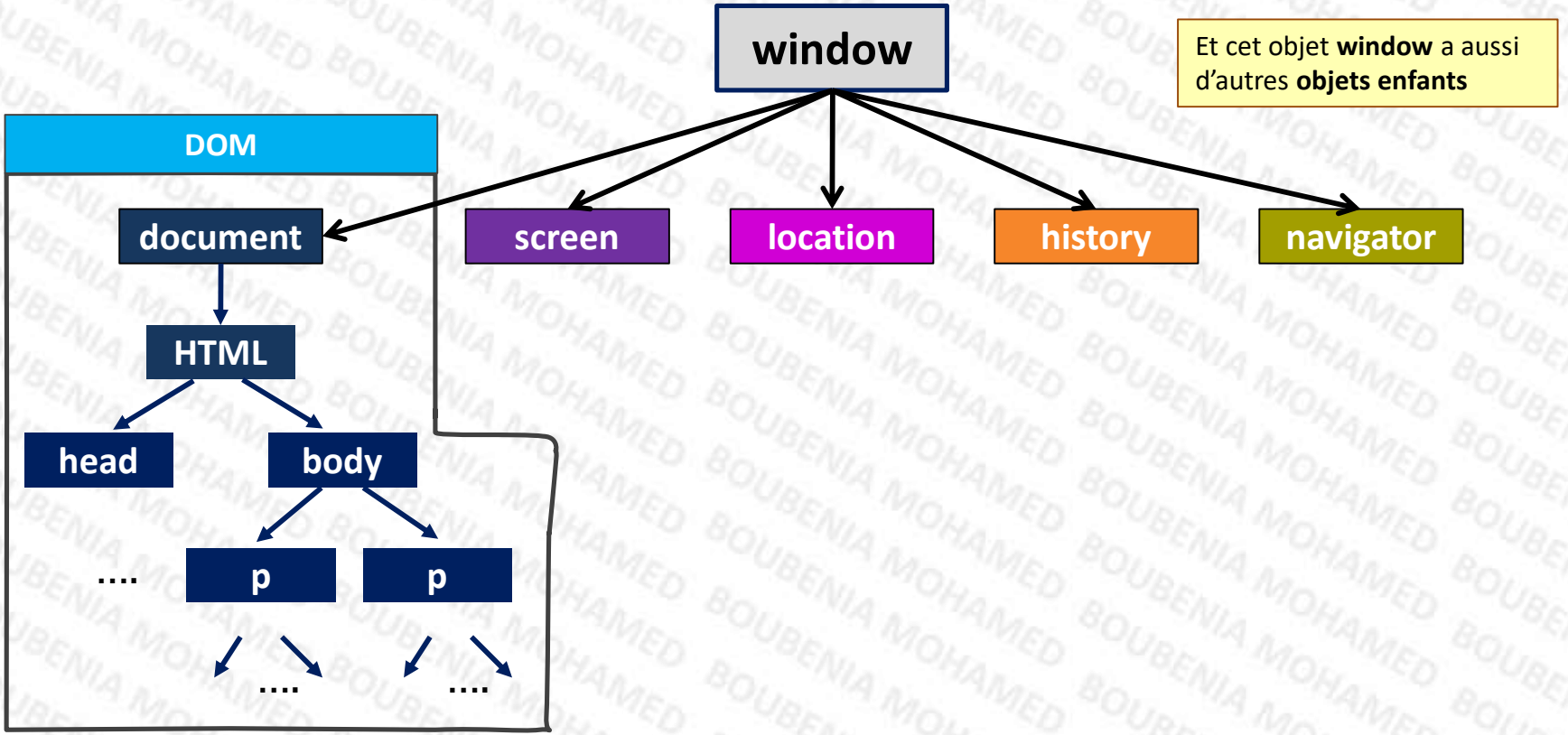
5- Le BOM (Browser Object Model)

Maintenant, il faut savoir que l'objet **document** est lui-même un enfant de l'objet **window**



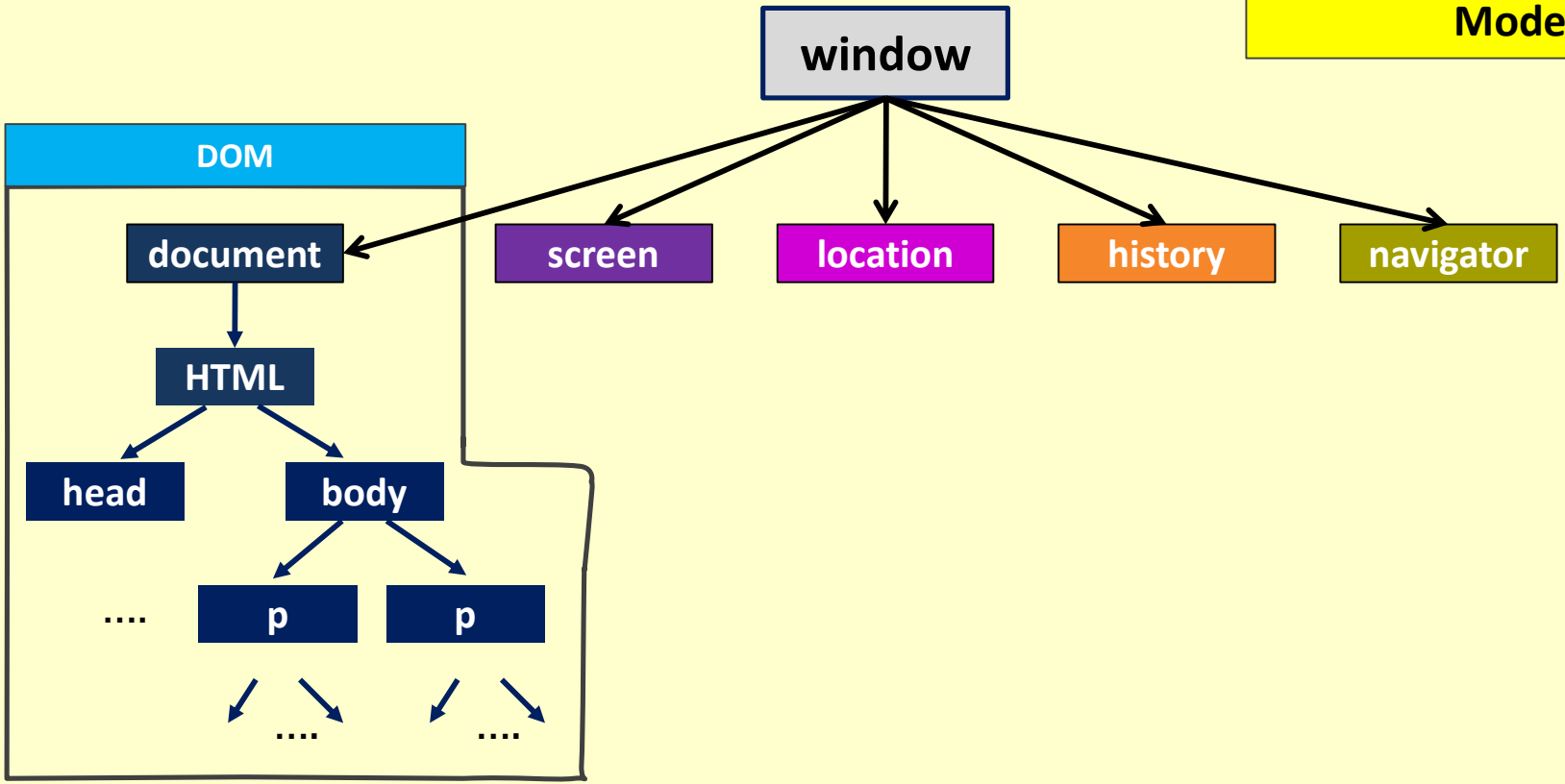


5- Le BOM (Browser Object Model)



5- Le BOM (Browser Object Model)

BOM: Browser Object Model



5- Le BOM (Browser Object Model)

Quelques propriétés et méthodes intéressantes

Voir d'autres ici:
https://www.w3schools.com/jsref/obj_window.asp

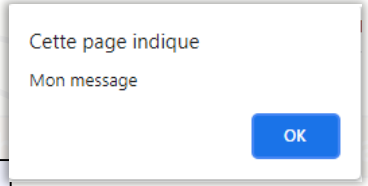
window

Méthodes

alert()

Affiche une boite d'alerte.

```
window.alert("Mon message");
// ou simplement
alert("Mon message");
```

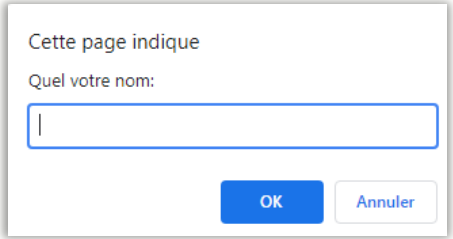


Pop-ups

Prompt()

Affiche une boite avec champs de saisie.

```
var nom = prompt("Quel votre nom:");
```



scrollTo()

Scroll jusqu'à une position précise

```
// scroll à la position: x = 500px horizontal y = 20px vertical
scrollTo(500, 20);
```

scrollBy();

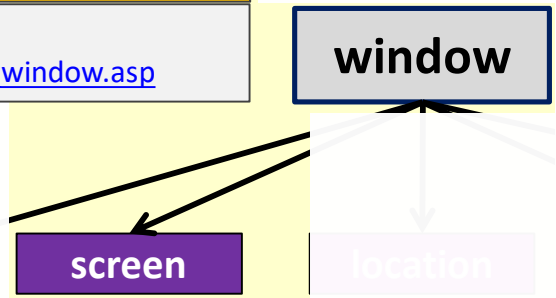
Scroll par un pas spécifié

```
// scroll de 500px horizontal et 20px vertical
scrollBy(500, 20);
```

5- Le BOM (Browser Object Model)

Quelques propriétés et méthodes intéressantes

Voir d'autres ici:
https://www.w3schools.com/jsref/obj_window.asp



Propriétés

- Height/width
- Orientation

```
// La hauteur de l'écran
screen.height;

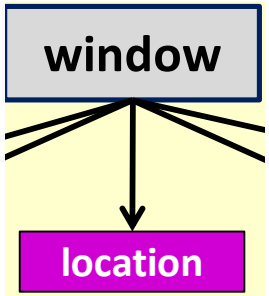
// La largeur de l'écran
screen.width;

// L'orientation de l'écran (landscape/portrait)
screen.orientation;
```

5- Le BOM (Browser Object Model)

Quelques propriétés et méthodes intéressantes

Voir d'autres ici:
https://www.w3schools.com/jsref/obj_window.asp



Méthode

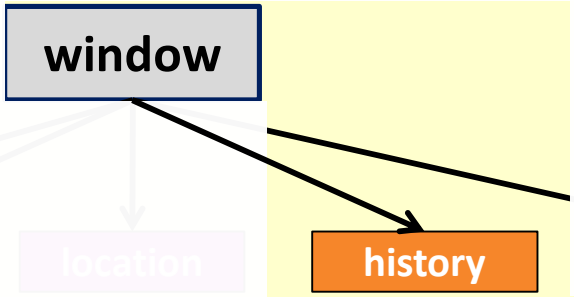
href()

```
// Donne l'URL courante
console.log(location.href);
// Changer l'URL (fait une redirection)
location.href="http://www.usthb.dz";
```

5- Le BOM (Browser Object Model)

Quelques propriétés et méthodes intéressantes

Voir d'autres ici:
https://www.w3schools.com/jsref/obj_window.asp



Méthodes

- back()
- go()
- forward()

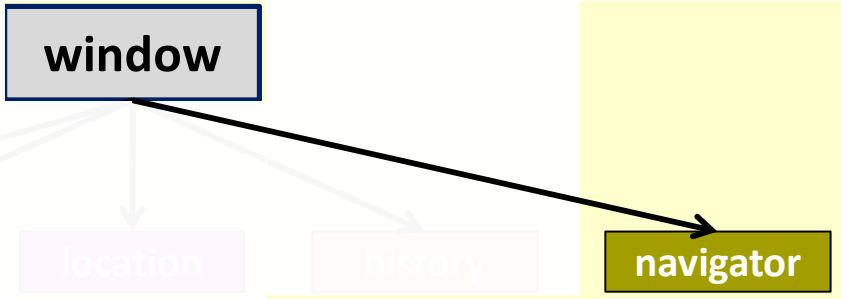


```
// même effet que de cliquer sur "précédent" sur le navigateur
history.back();
// retroune 2 fois en arrière dans l'historique
history.go(-2);
// même effet que de cliquer sur "avant" (contraire de précédent)
history.forward();
```

5- Le BOM (Browser Object Model)

Quelques propriétés et méthodes intéressantes

Voir d'autres ici:
https://www.w3schools.com/jsref/obj_window.asp



Propriétés

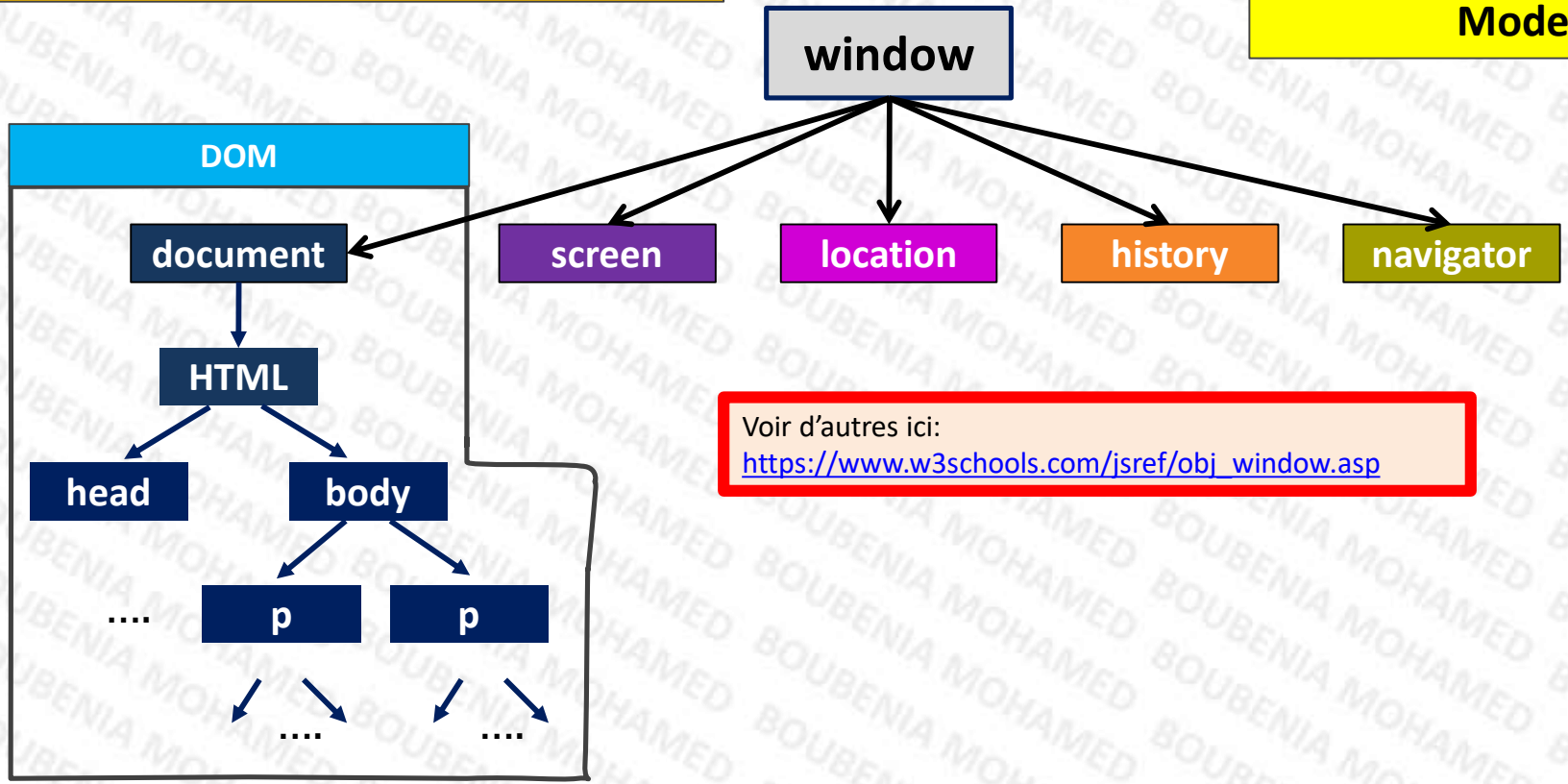
platform

```
// le système d'exploitation
// Exemple: "Win32", "MacIntel", "FreeBSD i386", "WebTV OS"
console.log(navigator.platform);
```

5- Le BOM (Browser Object Model)

Quelques propriétés et méthodes intéressantes

BOM: Browser Object Model



Voir d'autres ici:
https://www.w3schools.com/jsref/obj_window.asp

Les APIs Natives JavaScript

BOUBENIA MOHAMED

6- Les APIs natives de Javascript

Les navigateurs modernes offrent une gamme d'APIs (méthodes, objets , propriétés) permettant d'étendre les capacités de Javascript, en vue de construire des applications riches.

- Géolocalisation
- Storage
- ML/IA
- Web APIs
- BOM
- Réseau
- DOM
- Sécurité
- Performance

```
// Localistion (coordonnées GPS)
navigator.geolocation.getCurrentPosition(pos => {
  console.log(pos.coords.latitude, pos.coords.longitude);
});

// Stockage de données dans le navigateur
localStorage.setItem("nom", "Mohamed");
console.log(localStorage.getItem("nom"));

// WebCam+Son
navigator.mediaDevices.getUserMedia({ video: true, audio:true })
  .then(stream => {
    document.querySelector("video").srcObject = stream;
  })
  .catch(err => {
    console.error("Erreur d'accès caméra :", err);
  });
```

Voir d'autres ici:
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Client-side_APIs/Introduction

Manipulation du DOM

Il est important de comprendre le DOM, pour savoir comment sélectionner un élément HTML et comment le modifier.

Nous allons décortiquer

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

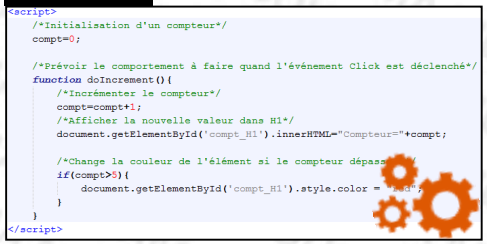
Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Nous allons décortiquer

Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Nous allons décortiquer

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

Script JS

Les Evénements

Il est important de connaitre les événements possibles, pour maitriser l'interaction avec l'utilisateur

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>
            Incrémenter
        </button>
    </body>
</html>
```



6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>
            Incrémenter
        </button>
    </body>
</html>
```



le "lorsque l'utilisateur clique"

C'est un Evénement

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>
            Incrémenter
        </button>
    </body>
</html>
```



onclick

le "lorsque l'utilisateur clique"

C'est un Evénement

En JavaScript, cet événement s'appel "onclick"

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>
            Incrémenter
        </button>
    </body>
</html>
```



onclick

Mais je le déclare où ??

le "lorsque l'utilisateur clique"

C'est un Evénement

En JavaScript, cet événement s'appel "onclick"

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick    >
            Incrémenter
        </button>
    </body>
</html>
```

onclick



le "lorsque l'utilisateur clique" sur sur le bouton "Incrémenter"

C'est un Evénement

En JavaScript, cet événement s'appel "onclick"

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick   >
            Incrémenter
        </button>
    </body>
</html>
```



Mais je n'ai pas terminé, car je n'ai pas encore précisé ce que j'ai envie de faire quand l'événement est déclenché

le | "lorsque l'utilisateur clique" | sur | sur le bouton "Incrémenter"

C'est un | Evénement

En JavaScript, cet événement s'appel "onclick"

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```



le | "lorsque l'utilisateur clique" | sur | sur le bouton "Incrémenter" | Je fait un traitement | Une fonction qu'on appellera par exemple "do Traitement", et qu'on définira

C'est un | Evénement

En JavaScript, cet événement s'appel "onclick"

compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Rappelez-vous

HTML

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```



Script JS

```javascript
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```

Une fonction qu'on appellera par exemple "do Traitement", et qu'on définira

compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

6- Les Evénements

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", (un traitement)
(traitement) c'est : le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

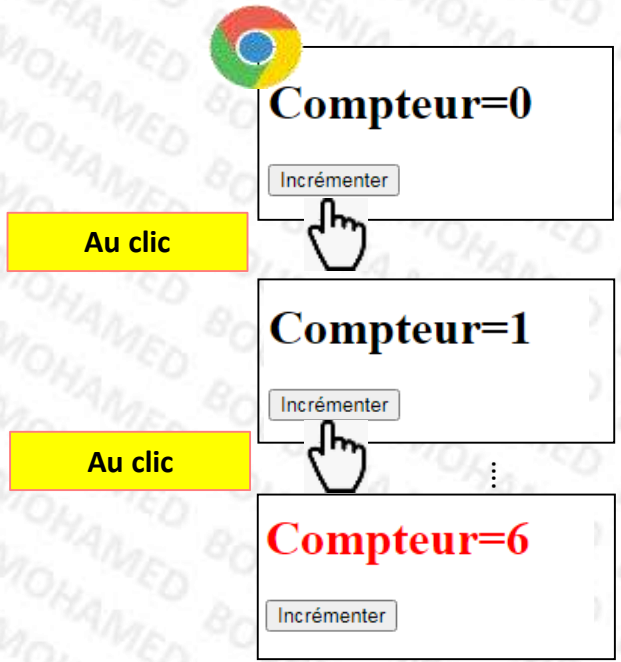
```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



6- Les Evénements

Autres notations

```
<button onClick="sayHello()">Action</button>
<script>
    // Fonction à appeler
    function sayHello(){
        alert("Hello !");
    }
</script>
```

```
<button>Action</button>
<script>
    // Element bouton
    btn= document.getElementsByTagName("button")[0];

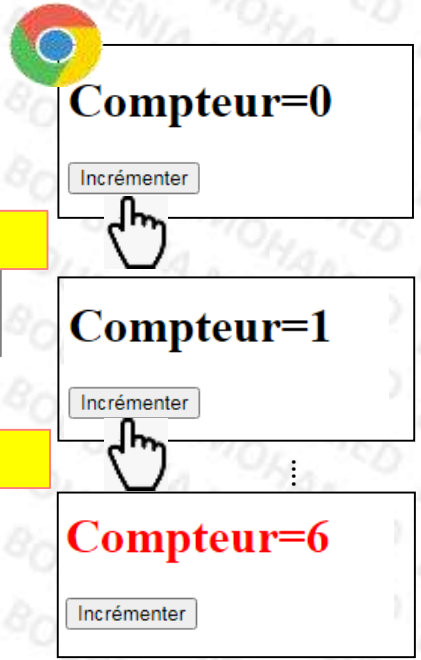
    // Attacher un évenement (click)
    // qui exécute fonction anonyme
    btn.addEventListener("click", function (){
        sayHello();
    });

    // Attacher un évenement (click)
    // qui exécute la fonction sayHello
    /*
    btn.addEventListener("click", sayHello);
    */
</script>
```

6- Les Evénements

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onmouseover="doTraitement()">
            Incrémenter
        </button>
    </body>
</html>
```



Au survol

Passage de la souris, sans clique

Au survol

le "lorsque l'utilisateur clique

Evénement survol

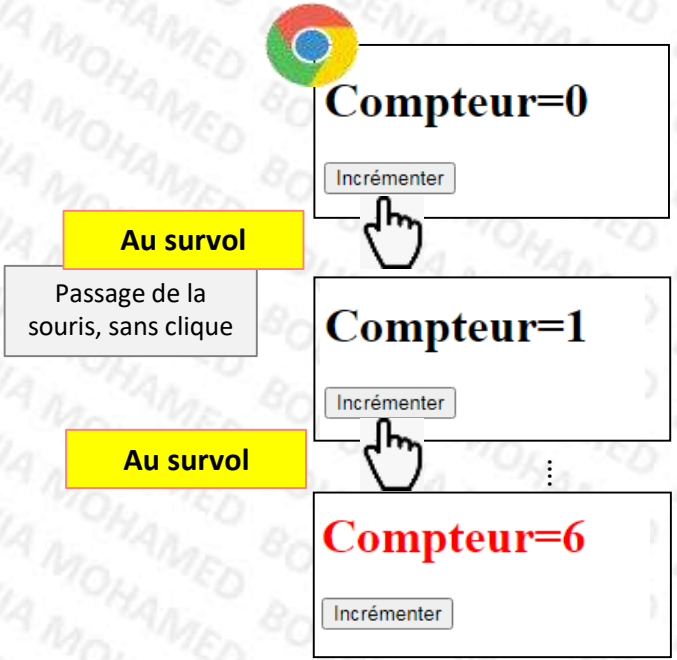
`onmouseover`

Mais il existe d'autres événements possibles

6- Les Evénements

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onmouseover="doTraitement()">
            Incrémenter
        </button>
    </body>
</html>
```



le "lorsque l'utilisateur clique

Evénement survol

onmouseover

Liste des événements possibles

https://www.w3schools.com/jsref/dom_obj_event.asp

https://developer.mozilla.org/fr/docs/Web/Events

6- Les Evénements

Event	Description	Exemple
dbclick	Double-clic sur un élément.	`div.addEventListener("dblclick", ()=> console.log("Double clic"));`
keydown	Une touche du clavier est enfoncée.	`document.addEventListener("keydown", e => console.log(e.key));`
keyup	Une touche du clavier est relâchée.	`document.addEventListener("keyup", ()=> console.log("Relâché"));`
focus	Élément reçoit le focus (cliqué ou tabulé).	`input.addEventListener("focus", ()=> input.style.background="yellow");`
blur	Élément perd le focus.	`input.addEventListener("blur", ()=> input.style.background="white");`
load	Page ou ressource terminée de charger.	`window.addEventListener("load", ()=> console.log("Page chargée"));`
contextmenu	Clic droit (menu contextuel).	`div.addEventListener("contextmenu", e => { e.preventDefault(); alert("Clic droit !"); });`
dragstart	Début d'un glisser-déposer.	`img.addEventListener("dragstart", ()=> console.log("Drag start"));`
drop	Élément déposé.	`zone.addEventListener("drop", e => console.log("Déposé !"));`

6- Les Evénements

HTML

```
<body>
    <h1
        onmouseover="document.body.style.backgroundColor = 'red';"
        onmouseleave="document.body.style.backgroundColor = 'white';"
    >Titre 1</h1>
    <h2>Titre 2<h2>
</body>
```

Ici: j'ai mis mon traitement directement dans le HTML (juste montrer la possibilité).
J'ai aussi utilisé 2 événement:
- onmousover (au passage de la souris)
- onmousleave (quand l'élément n'est plus survolé)



Au survol de Titre1



Quand Titre1 n'est plus survolé



3- Les Bases du Langage JavaScript

Bonnes pratiques

Structure et Organisation:

- Utiliser let et const à la place de var pour mieux gérer la portée.
- Séparer les responsabilités : logique, affichage (DOM), et style (CSS).
- Utiliser des classes et des objets littéraux pour structurer les données.
- Documenter le code.

Lisibilité du code:

- Utiliser === au lieu de == (comparaison stricte).
- Favoriser les fonctions pures (sans effet de bord) (autonome)
- Préférer la déstructuration (const {x, y} = obj) pour clarifier les extraits de données.
- Utiliser les template literals (`Bonjour ${nom}`) pour la lisibilité des chaînes.
- Écrire du code concis avec les fonctions fléchées quand approprié.
- Aérer le code (indentation, espaces, retours à la ligne).
- Utiliser Prettier ou un formateur automatique.

Performance:

- Préférer les méthodes natives de tableau (map, filter, reduce) aux boucles manuelles lourdes.
- Utiliser l'asynchrone moderne (async/await) pour éviter le blocage du thread principal.
- Éviter la manipulation directe et fréquente du DOM (regrouper les mises à jour).
- Utiliser des outils de profiling (Performance tab, Lighthouse) pour détecter les goulots d'étranglement.
- Éviter les variables globales (risque de surcharge mémoire et conflits)

Organisation et maintenance:

- Travailler avec un contrôle de version (Git) et des revues de code.
- Respecter le mode strict ("use strict";) pour éviter les erreurs silencieuses.
- Isoler la configuration et les variables d'environnement.

https://developer.mozilla.org/en-US/docs/MDN/Writing_guidelines/Code_style_guide/JavaScript

3- Les Bases du Langage JavaScript

Evolution du langage

Evolution

1995: Naissance.

1997-1999: ES1 à ES3: Première version stable

2009: ES5: Modernisation (strict mode, JSON, forEach, map, reduce) version supporté par tous les navigateurs

2015: ES6 Révolution (let, const, class, modules, fonctions fléchées, Promise,…)

2016-2025: ES7-ES15: Modernisation continue (async/await, rest/spread, flat, chaining…)

Où se dirige JS ?

Langage universel: côté client/serveur/mobile, IA, IOT
Interopérabilité
Performance/typage: apparition de TypeScript pour typer le code.

Abandonce de Frameworks

- **Front-end**: React, Vue, Angular…
- **Back-end:** NodeJS, Express, NestJS

BOUBENIA MOHAMED

Liens utiles



W3Schools
https://www.w3schools.com/js



Mozilla Developer Network
https://developer.mozilla.org/en-US/docs/Web/JavaScript



JAVASCRIPT.INFO
https://fr.javascript.info/



Spécification ECMAScriot 2026
https://tc39.es/ecma262/